Pontifícia Universidade Católica do Rio Grande do Sul — Escola Politécnica
46504-04 Construção de Software — Turma 330 — 2026/2
Prof. Michael da Costa Mora

PulsePass

Plataforma Distribuída de Bilhetagem, Reserva e Acesso para Megaeventos

Pacote 1 — Processo de Desenvolvimento e Requisitos

Grupo 9
Gabriel Gerhardt de Marque
Mateus Granja de Andrade Madalosso Tieppo
Filipe Tesche Roman

Repositório: **https://github.com/Mateus-Tieppo/construcao-de-software**

Porto Alegre, 14 de outubro de 2026

Sumário

1. Descrição do Processo de Desenvolvimento Suportado por IA
2. Documento de Visão do Produto
3. Backlog com Épicos e Histórias de Usuários no Formato de Story Mapping
4. Requisitos Não-Funcionais
5. Mockups (medium res) e Mapa de Navegação do Front-end

Os arquivos-fonte (Markdown, diagramas Mermaid e mockups HTML) são versionados no repositório do grupo e serão atualizados nos Pacotes 3 e 4.

1. Descrição do Processo de Desenvolvimento Suportado por IA

> Base conceitual: aula 09 — "A Transição para o SDD" (Prof. Michael Mora). Este item descreve o processo que o Grupo 9 adota no trabalho e que **já foi aplicado na produção deste Pacote 1**.

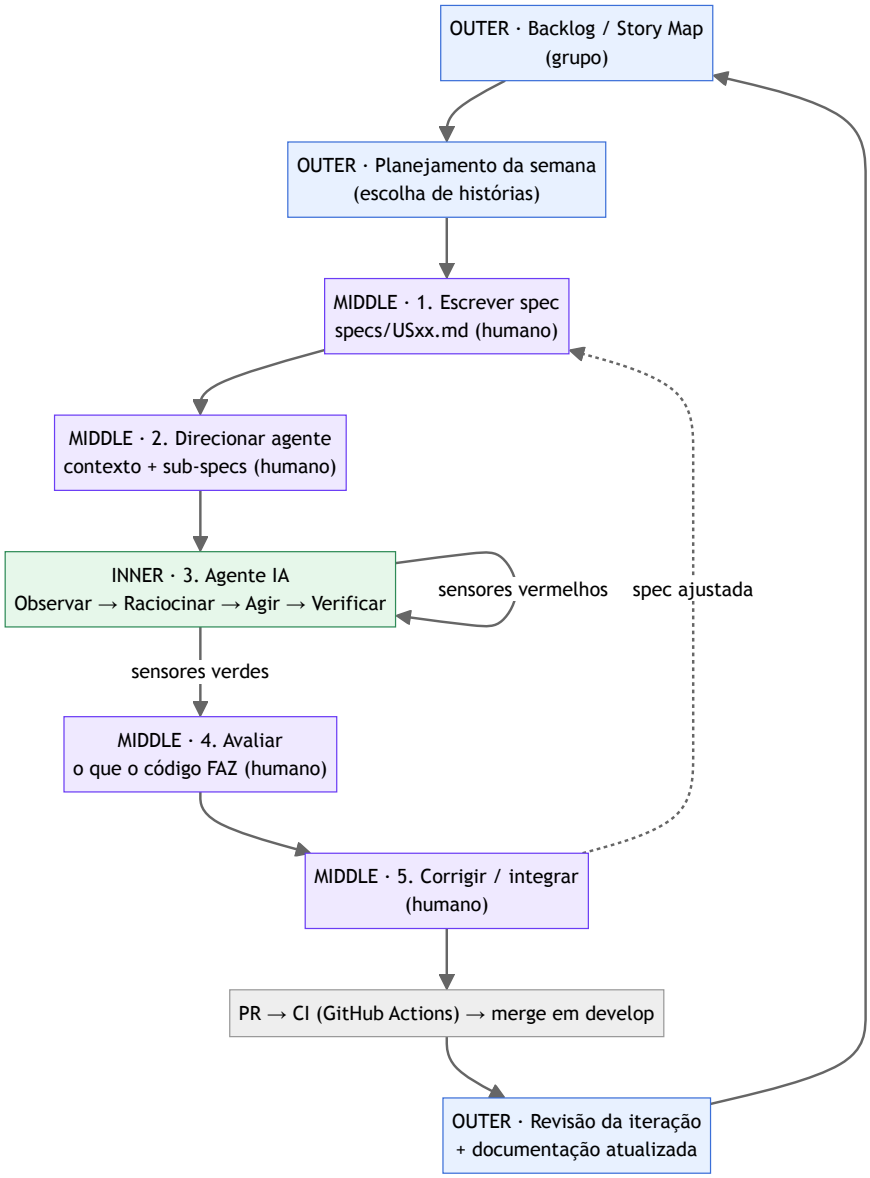
1.1 Visão geral do processo

O Grupo 9 adota **Spec-Driven Development (SDD) com agentes de codificação supervisionados**. A **especificação é a fonte única de verdade**: toda história do backlog vira uma spec versionada no repositório antes de qualquer código; o código (e os testes) são derivados da spec por agentes de IA e **validados contra os critérios de verificação dela**, por sensores automáticos e por revisão humana.

O processo é iterativo e alinhado aos pacotes do enunciado:

Iteração	Período	Objetivo	Saída principal
I1 — Pacote 1	até 14/10/2026	Processo e requisitos	Visão, Story Map, RNFs, mockups, este documento
I2 — Pacote 2	15/10 → 04/11	Arquitetura e infraestrutura	C4 (níveis 1–3), modelo de dados, tópicos Kafka, pipeline, GCS; esqueleto dos serviços + docker-compose + CI
I3 — Pacote 3	05/11 → 23/11	Checkpoint (MVP)	Histórias da fatia MVP implementadas e testadas
I4 — Pacote 4	24/11 → 02/12	Produto completo	Fatia R2, testes de carga, documentação atualizada

Dentro das iterações I2–I4 trabalhamos em **ciclos semanais** (planejamento na segunda-feira, revisão/integração na quarta, aproveitando as aulas de trabalho).



1.2 Ciclo de desenvolvimento

O ciclo segue os três loops da aula 09:

- **Outer loop (governança)** — o grupo mantém o Story Map (item 3), define a fatia da iteração e atualiza a documentação ao fim de cada pacote. Decisões de arquitetura ficam registradas em ADRs (`docs/adr/`).
- **Middle loop (engenharia supervisória — humanos)** — cada integrante, como "dono" de uma história:
  - **Direcionamento:** decompõe a história em blocos do tamanho de um agente, escreve/ajusta a spec e escolhe o contexto (3–5 arquivos relevantes);
  - **Avaliação:** revisa o que o código realmente faz, caçando "alucinações plausíveis" (ex.: lock que não é atômico, consumidor que não é idempotente);
  - **Correção:** integra saídas de agentes paralelos e reconduz à consistência com a spec.
- **Inner loop (agente)** — o agente de codificação lê a spec, gera código e testes, executa compilador/testes/linters e se corrige até os sensores ficarem verdes.

Fases de uma história (fluxo padrão):

#	Fase	Responsável	Artefato / portão
1	Seleção da história no Story Map	Grupo	Issue no GitHub ligada à US
2	Spec da história	Humano (com rascunho por IA)	`specs/USxx-nome.md` aprovada por outro integrante
3	Decomposição em sub-specs	Agente coordenador + humano	Sub-specs por camada/serviço, sem arquivos em comum
4	Implementação	Agentes implementadores	Branch `feature/USxx-...` com código + testes

#	Fase	Responsável	Artefato / portão
5	Verificação	Sensores + agente verificador	Testes, cobertura, ArchUnit, relatório do juiz
6	Pull Request	Humano	PR para `develop` com checklist da spec
7	CI	GitHub Actions	Lint + testes + build verdes (obrigatório)
8	Revisão e merge	Outro integrante (humano)	1 aprovação humana + CI verde → merge

1.3 Ferramentas

Ferramenta	Uso no processo
Claude Code (Anthropic, terminal/VS Code)	Agente de codificação principal: geração de specs, código, testes, documentação e diagramas; subagentes paralelos; leitura automática do `CLAUDE.md`; hooks
Antigravity (AGY CLI)	Alternativa vista no codelab da aula 09, usada para comparação e por integrantes que preferirem **[Decisão do grupo]**
VS Code	IDE comum, com extensão do Claude Code
GitHub	Repositório, Issues (backlog), Projects (quadro da iteração), Pull Requests, proteção de branches
GitHub Actions	CI: lint, testes unitários e de integração (Testcontainers), cobertura (JaCoCo), mutação (PIT), build de imagens; revisão por IA no PR
Docker / Docker Compose	Ambiente local completo e reprodutível (RNF37)
Java 21, Spring Boot 3, Spring Cloud, Maven	Back-end dos microsserviços
React + TypeScript (Vite)	Front-end SPA
JUnit 5, Mockito, Testcontainers, ArchUnit, PIT, k6	Sensores computacionais (testes, arquitetura, mutação, carga)
Checkstyle/Spotless, ESLint/Prettier, gitleaks	Linters e varredura de segredos
Mermaid	Diagramas como código (story map, navegação, ciclo, FSM, C4 no Pacote 2)
HTML/CSS + Chrome headless	Mockups versionáveis (item 5) e geração do PDF de entrega

1.4 Agentes e papéis

Aplicamos o **Padrão de Agente Adversarial**:

Papel	Quem	Responsabilidade	Entrada → Saída
Coordenador (Tech Lead)	Sessão principal do Claude Code, supervisionada pelo dono da história	Lê a spec raiz e gera sub-specs independentes (ex.: domínio/FSM, adaptador de persistência, consumidor Kafka, tela React)	`specs/USxx.md` → `specs/USxx/*.md`
Implementadores	Subagentes do Claude Code (em paralelo) ou sessões separadas de cada integrante	Implementam uma sub-spec cada, com testes	Sub-spec → código + testes na branch
Verificador (Juiz)	Subagente separado com prompt de "revisor adversarial" (sem acesso ao raciocínio do implementador) + revisão de PR por IA no CI	Avalia **estritamente contra os critérios de verificação** da spec e procura violações (overselling, falta de idempotência, segredo exposto, transição de FSM inválida)	Diff + spec → lista de violações
Supervisor humano	Integrante dono da história; outro integrante revisa o PR	Direciona, avalia e corrige; decide o merge. **Nenhum código vai para `develop` sem revisão humana**	—

Como evitamos colisão de arquivos:

- Cada sub-spec declara explicitamente os **arquivos/pacotes que pode alterar**; o coordenador só gera sub-specs com conjuntos disjuntos.
- Arquitetura hexagonal por serviço (domínio, aplicação, adaptadores) facilita a divisão.
- Cada integrante trabalha em sua `feature/*`; integração apenas via PR.
- Divisão inicial de responsabilidade por módulo **[Decisão do grupo — a confirmar na I2]**: Gabriel — IAM + Telemetria/DevOps; Mateus — Inventário + Checkout; Filipe — Ingressos/Notificações + Front-end. Revisões de PR são cruzadas.

1.5 Specs utilizadas

Local: pasta `specs/` na raiz do repositório, uma spec por história (`specs/US18-reservar-assento.md`), sub-specs em subpasta (`specs/US18/`). Specs são versionadas e revisadas em PR como código.

Nível de maturidade de SDD escolhido: "spec-anchored" (intermediário) — a spec é escrita antes, permanece no repositório e é **atualizada junto com o código** quando o comportamento muda; não chegamos a "spec-as-source" (gerar todo o código só a partir da spec, sem editar código) porque o não-determinismo do LLM e o prazo exigem ajustes manuais. Dosamos o peso: bugfix pequeno não

exige spec completa (só issue + teste que reproduz); histórias de risco alto (concorrência, pagamento, segurança) exigem spec completa e verificador.

Template (6 elementos):

```
# USxx — <título>
## 1. Objetivo            — o que e por quê (liga à história do Story Map)
## 2. Contexto            — serviço, arquivos relevantes, decisões/ADRs aplicáveis
## 3. Restrições          — regras inegociáveis (ex.: zero overselling, sem escrita direta no Kafka)
## 4. Interfaces/Contratos — endpoints (OpenAPI), eventos Kafka (esquema), tabelas
## 5. Critérios de aceitação e verificação — Dado/Quando/Então + testes obrigatórios + métricas
## 6. Fora de escopo      — o que o agente NÃO deve fazer
```

Seguindo a mitigação de "*lost in the middle*", as **restrições inegociáveis ficam no início** da spec e os **critérios de verificação/testes no fim**.

Exemplo — `specs/US18-reservar-assento.md` (resumida):

> **1. Objetivo.** Permitir que o Cliente reserve de 1 a 6 assentos atomicamente por 10 minutos (US18, US19). **2. Contexto.** `inventory-service`; pacote `domain.reserva`; Redis para lock com TTL; PostgreSQL como fonte da verdade; ADR-003 (estratégia de lock). **3. Restrições.** Tudo-ou-nada; zero overselling (RNF01); TTL exato de 600 s; devolução atômica na expiração; evento `ReservaExpirada` publicado **somente via Outbox**; domínio sem dependência de Spring (ArchUnit). **4. Interfaces.** `POST /eventos/{id}/reservas` (header `Idempotency-Key`, corpo `{assentos:[...]}`) → `201 {reservaId, expiraEm}` · `409` assento indisponível · evento `inventario.reserva-expirada.v1 {eventId, reservaId, assentos[], ocorridoEm}`. **5. Verificação.** (a) teste de integração com Testcontainers: 50 threads no mesmo assento → exatamente 1 sucesso; (b) teste com `Clock` controlado: expira em 600 s ± 5 s; (c) falha injetada no 3º assento → nenhum bloqueado; (d) cobertura de domínio ≥ 70%; (e) juiz confirma ausência de escrita direta no Kafka. **6. Fora de escopo.** Fila virtual; pagamento; WebSocket (US20).

1.6 Harness: controles feedforward e feedback

Agente = LLM + harness. O harness do grupo:

Controles feedforward (preventivos)

- **`CLAUDE.md` na raiz** (constituição do projeto), lido automaticamente no início de cada sessão: regras inegociáveis no topo (idioma, requisitos do domínio, proibição de segredos, pastas que não podem ser alteradas), mapa de pastas, tarefa atual e critérios de verificação no fim.
- **Progressive disclosure:** o `CLAUDE.md` é enxuto e aponta para documentos específicos (plano do pacote, enunciado, specs) em vez de colar tudo no contexto.
- **Skills / comandos reutilizáveis** em `.claude/` (ex.: `/nova-spec` gera spec a partir do template; `/verificar` roda o juiz contra a spec; `/pacote1` iniciou este pacote).
- **Templates e convenções:** template de spec, de PR (checklist com critérios da spec) e de ADR; convenção de commits (*Conventional Commits*).
- **Hooks do Claude Code:** bloquear edição de arquivos protegidos (`contexto/`, `.env`) e rodar formatador após edições.

Controles feedback (sensores)

- **Computacionais:** compilação; testes unitários da FSM e regras de domínio (**≥ 70% de cobertura de domínio**, JaCoCo como *gate*); testes de integração com **Testcontainers** (PostgreSQL, Kafka, Redis); **ArchUnit** (fitness functions: domínio sem framework, sem acesso a banco de outro serviço, sem `KafkaTemplate` em caso de uso); **PIT** (mutação ≥ 70%); linters; **gitleaks**; k6 (carga).
- **Inferenciais: LLM-as-a-judge** — subagente verificador local e revisão automática de PR por IA no GitHub Actions, avaliando aderência à spec, nomenclatura, cobertura dos critérios e segurança. O parecer do juiz é consultivo; o portão final é humano.

1.7 Integração com GCS e CI/CD

(Detalhamento completo do GCS no Pacote 2.)

- **Branches (Git Flow simplificado):** `main` (versões entregues por pacote, com *tag* `pacote-N`), `develop` (integração), `feature/USxx-descricao`, `fix/...`, `docs/...`.
- **Regras:** proibido *push* direto em `main` e `develop`; todo merge via **Pull Request** com **CI verde** + **1 aprovação humana** de outro integrante; PR referencia a issue e a spec; *squash merge* em `develop`.
- **CI (GitHub Actions)** a cada PR e merge em `develop`: lint → testes unitários + cobertura → testes de integração (Testcontainers) → ArchUnit → build das imagens → revisão por IA (comentário no PR).
- **CD:** merge em `main` gera imagens versionadas (GHCR) e o `docker-compose.yml` de demonstração.

- **Specs e documentação** passam pelo mesmo fluxo de PR — mudança de comportamento sem atualização da spec é motivo de reprovação no review.

1.8 Riscos do uso de IA e mitigação

Risco	Mitigação no processo
Lost in the middle / contexto longo	`CLAUDE.md` enxuto; regras no início e verificação no fim; contexto filtrado (3–5 trechos); revelação progressiva (um entregável/sub-spec por vez)
Alucinação plausível (código que parece certo mas viola concorrência/idempotência)	Critérios de verificação objetivos na spec; testes de integração reais; agente juiz adversarial; revisão humana focada no comportamento
Débito cognitivo e de intenção (*Triple Debt*)	Dono humano por história, que precisa explicar o código no review; specs e ADRs registram a intenção
Segredos em código	Regra no `CLAUDE.md`; `.env` fora do Git; gitleaks no CI; hooks bloqueando edição de `.env`
Não-determinismo (mesma spec → código diferente)	Sensores determinísticos como portão; testes baseados em propriedades para a FSM (jqwik)
Copy-paste / duplicação	Revisão de PR, linters e análise estática
Custo de tokens	Rigor proporcional ao risco; subagentes só para histórias grandes; reuso de skills
Dependência excessiva da ferramenta	Todo integrante deve conseguir rodar, testar e explicar qualquer parte do sistema sem a IA

1.9 Evidência: como este Pacote 1 foi produzido

O próprio Pacote 1 seguiu o processo descrito acima:

1. **Spec raiz:** `CLAUDE.md` (regras inegociáveis + critérios de verificação) e `pacote-1/PLANO-PACOTE-1.md` (tabela "Pronto quando…" por entregável), escritos pelo grupo a partir do enunciado (`contexto/01-enunciado-projeto-final.md`, tratado como fonte de verdade e protegido contra alteração).
2. **Direcionamento:** `pacote-1/PROMPT-INICIAL.md` definiu a ordem (Visão → RNFs → Story Map → Mockups → Processo) — revelação progressiva, um entregável por vez.
3. **Inner loop:** o Claude Code (VS Code) gerou cada documento, os mockups HTML e os diagramas Mermaid.
4. **Verificação adversarial** após cada entregável, item a item contra o plano (ex.: a verificação encontrou e corrigiu soma incorreta de pontos no backlog e uma legenda de módulo inexistente nos RNFs; ajustou anotações que sobrepunham conteúdo nos mockups).
5. **Supervisão humana:** o grupo revisou o conteúdo, validou as decisões marcadas como **[Decisão do grupo]** e gerou o PDF de entrega.

2. Documento de Visão do Produto

> Versão 1.0 — 14/10/2026 · Grupo 9 · Será atualizado nos Pacotes 3 e 4. Itens marcados com **[Decisão do grupo]** são extrapolações nossas sobre o enunciado.

2.1 Introdução e propósito

Este documento descreve a visão do produto a ser construído no Projeto Integrador de Construção de Software (PUCRS 2026/2): uma **plataforma distribuída de bilhetagem, reserva e acesso para megaeventos**. Ele define o problema, os usuários, o escopo funcional (cinco módulos), as restrições técnicas e os critérios de sucesso. É a referência para o Backlog (item 3), os Requisitos Não-Funcionais (item 4) e os Mockups (item 5).

2.2 Nome do produto

PulsePass — nome sugerido no enunciado e adotado pelo grupo **[Decisão do grupo]**. A ideia de "pulso" remete ao pico de tráfego na abertura das vendas e ao acompanhamento em tempo real; "pass" remete ao ingresso/acesso.

2.3 Problema

Aspecto	Descrição
O problema	Na abertura de vendas de megaeventos (shows, festivais, finais esportivas), dezenas de milhares de compradores chegam no mesmo instante (*flash crowd*). Sistemas baseados só em HTTP síncrono esgotam conexões de banco, ficam lentos ou caem, e falhas de concorrência vendem o mesmo assento duas vezes (*overselling*).
Afeta	Compradores (frustração, cobrança sem ingresso), organizadores (prejuízo financeiro e de imagem, reembolsos), operadores de acesso (fraude e filas na portaria).
Impacto	Perda de receita, reclamações em massa, ingressos duplicados ou falsificados, retrabalho manual de conciliação entre pagamentos e ingressos.
Uma solução bem-sucedida	Aguenta o pico com degradação controlada (fila/limitação), garante **zero overselling**, libera automaticamente assentos de reservas não pagas em **10 minutos**, emite ingressos seguros (PDF + QR Code) e dá visibilidade em tempo real a todos os envolvidos.

2.4 Oportunidade / posicionamento

Para organizadores de grandes eventos e seus públicos, **que** precisam vender milhares de ingressos em minutos sem erros, **o PulsePass é** uma plataforma de bilhetagem distribuída **que** combina bloqueio concorrente de assentos, checkout resiliente orientado a eventos e emissão assíncrona de ingressos. **Diferente de** sistemas monolíticos síncronos, **nosso produto** é baseado em microsserviços e mensageria (Kafka), isola o pico de leitura do catálogo do fluxo transacional e garante consistência entre banco e broker com o padrão *Transactional Outbox*.

2.5 Stakeholders

Stakeholder	Interesse	Responsabilidade no projeto
Prof. Michael da Costa Mora	Avaliar aplicação de arquitetura, testes, DevOps e processo com IA	Cliente acadêmico / avaliador
Grupo 9 (Gabriel, Mateus, Filipe)	Entregar produto e documentação de qualidade	Equipe de desenvolvimento (com agentes de IA supervisionados)
Organizadores de eventos	Vender sem overselling, acompanhar vendas	Usuário de negócio / fonte de requisitos
Compradores (público)	Comprar rápido e com segurança	Usuário final
Operadores de acesso	Validar ingressos na entrada	Usuário operacional
Administradores da plataforma	Governança, segurança, saúde do sistema	Usuário técnico/gestor
Gateway de pagamento (simulado)	Receber requisições de cobrança e notificar resultado	Sistema externo
Serviço de e-mail/SMS/push	Entregar comprovantes	Sistema externo

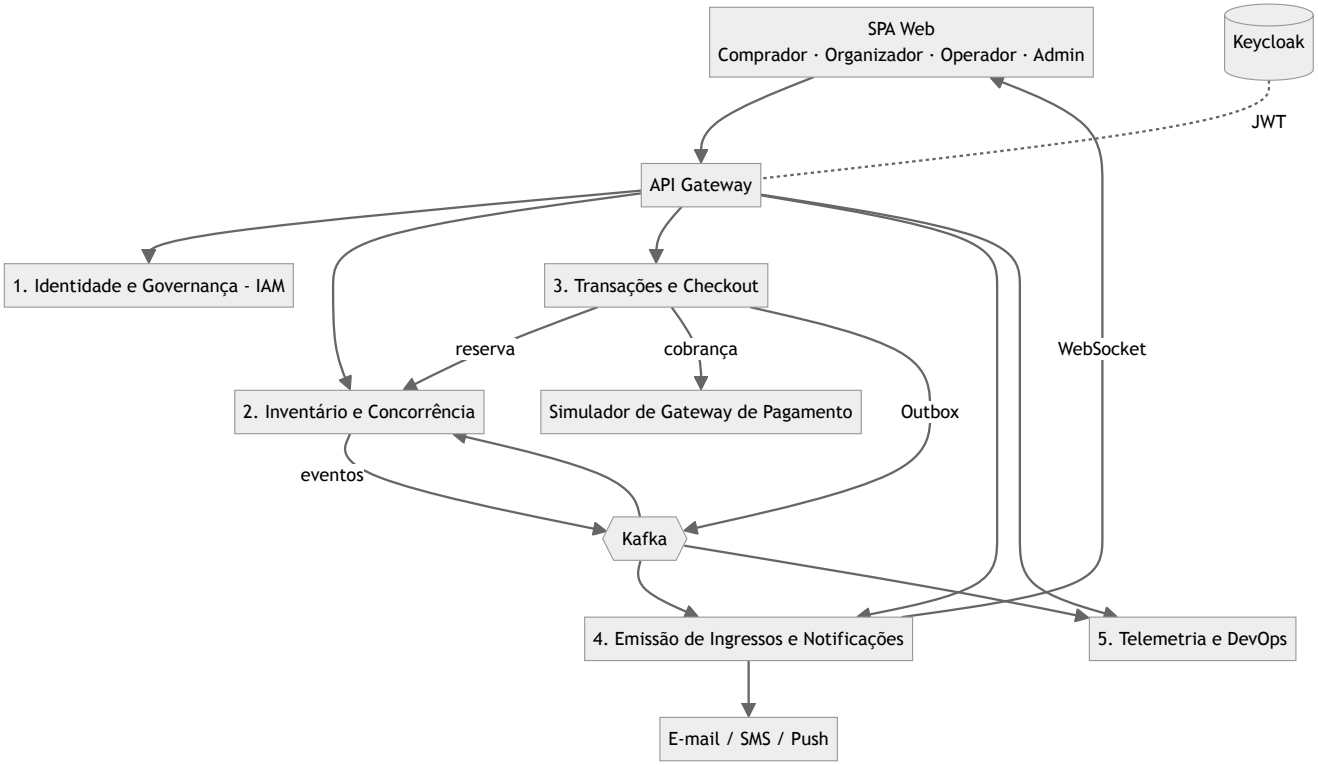
2.6 Usuários e perfis (RBAC)

Os perfis correspondem aos papéis RBAC exigidos pelo enunciado e são gerenciados no **Keycloak**.

Perfil	Quem é	Objetivos principais	Permissões resumidas
Cliente (Comprador)	Pessoa do público geral	Encontrar evento, escolher assento, pagar, receber ingresso	Ver catálogo, criar reserva/pedido, pagar, ver/baixar os próprios ingressos
Organizador	Produtora ou empresa que realiza o evento	Cadastrar eventos, lotes e mapas de assentos; acompanhar vendas	CRUD dos próprios eventos, lotes e mapas; ver relatórios dos próprios eventos
Operador	Equipe de portaria / suporte no local	Validar ingressos (QR Code) e apoiar o público	Validar check-in, consultar status de ingresso/pedido de eventos atribuídos
Admin	Equipe da plataforma	Aprovar organizadores, gerir usuários e papéis, auditar, monitorar	Gestão de usuários/papéis, aprovação de organizadores, trilha de auditoria, painel de telemetria

2.7 Visão geral do produto e módulos

O PulsePass é composto por uma **SPA** (front-end web) que consome APIs através de um **API Gateway**, e por microsserviços que se comunicam de forma síncrona (REST) apenas quando necessário e de forma assíncrona via **Kafka** nos demais casos.



#	Módulo	Responsabilidade	Destaques técnicos
1	**Identidade e Governança (IAM)**	Autenticação, autorização, cadastro e auditoria	Keycloak; JWT stateless assinado; refresh token em chave-valor (Redis); RBAC com 4 papéis; trilha de auditoria de credenciais; gestão cadastral de organizadores
2	**Inventário e Concorrência**	Eventos, lotes, mapas de assentos e bloqueio	Catálogo de alta leitura (cache); alocação e devolução **atômicas** de assentos; lock com TTL de 10 min; publica `ReservaExpirada` no Kafka
3	**Transações e Checkout**	Pedido e pagamento	FSM **Criado** → **Reservado** → **Aguardando** → **Pago / Expirado**; `Idempotency-Key` no header; **Transactional Outbox** em transação ACID (PostgreSQL) → Kafka; integração com simulador de gateway
4	**Mensageria e Despacho (Ingressos e Notificações)**	Emissão e entrega do ingresso	Consome `PagamentoConfirmado`; consumo idempotente; PDF com carimbo de tempo e **QR Code** assinado; envio multicanal; atualização de tela em tempo real via **WebSocket**
5	**Análise e Operações (Telemetria e DevOps)**	Métricas, painéis e automação	Ingestão de eventos de navegação e compra; painel em tempo real; Prometheus/Grafana; CI no GitHub Actions; ambiente completo em docker-compose

2.8 Funcionalidades principais

Comprador

- F01 — Cadastro/login (Keycloak) e recuperação de sessão via refresh token.
- F02 — Navegar no catálogo de eventos (busca, filtros por data/cidade/categoria).
- F03 — Visualizar mapa de assentos com disponibilidade atualizada em tempo real.
- F04 — Reservar assento(s) com contagem regressiva de 10 minutos.

- F05 — Pagar o pedido (simulador de gateway) com proteção contra clique duplo (Idempotency-Key).
- F06 — Receber ingresso em PDF com QR Code por e-mail e na área "Meus ingressos", com aviso em tempo real.

Organizador

- F07 — Solicitar cadastro como organizador (aprovado pelo Admin).
- F08 — Criar evento, lotes (preço, quantidade, janela de vendas) e mapa de assentos por setor.
- F09 — Publicar/encerrar vendas e acompanhar vendas por lote em tempo real.

Operador

- F10 — Validar ingresso por leitura de QR Code (válido, já utilizado, inválido).
- F11 — Consultar status de pedido/ingresso para suporte no local.

Admin

- F12 — Aprovar/bloquear organizadores e gerir usuários e papéis.
- F13 — Consultar trilha de auditoria de credenciais (logins, falhas, revogações, mudanças de papel).
- F14 — Painel de telemetria: vendas por minuto, reservas ativas/expiradas, latência, erros, lag do Kafka.

Transversais

- F15 — Expiração automática de reservas e devolução atômica do assento ao estoque.
- F16 — Pipeline de CI/CD e ambiente reproduzível em docker-compose.

2.9 Restrições e premissas

Restrições do enunciado (obrigatórias)

- R01 — **Zero overselling**: um assento nunca pode estar vinculado a dois pedidos pagos.
- R02 — **Reserva temporária expira estritamente em 10 minutos**, devolvendo o assento ao estoque.
- R03 — Estilo arquitetural de **microsserviços**.
- R04 — Provedor de identidade **Keycloak ou Cognito** → adotamos **Keycloak** (roda localmente em container, sem custo de nuvem) **[Decisão do grupo]**.
- R05 — **Kafka** como provedor de streaming.
- R06 — Tratar o **dilema da escrita dupla** (Transactional Outbox) e a **idempotência no consumo**.
- R07 — FSM do pedido com os estados **Criado, Reservado, Aguardando, Pago, Expirado**.
- R08 — Suporte a **Idempotency-Key** em cabeçalhos HTTP.
- R09 — Testes unitários das máquinas de estado com **≥ 70% de cobertura de domínio**; integração com **Testcontainers** (PostgreSQL, Kafka, Redis).
- R10 — Ambiente completo via **docker-compose.yml** (backend, frontend, bancos, Kafka e UI de mensageria).
- R11 — CI no **GitHub Actions** executando linters, testes e build a cada Pull Request ou merge na branch `develop`.
- R12 — Processo de desenvolvimento **suportado por IA** e documentado.

Premissas / decisões do grupo

- P01 — Back-end em **Java 21 + Spring Boot 3 + Spring Cloud** (Gateway, Config), coerente com os exercícios da disciplina **[Decisão do grupo]**.
- P02 — Front-end SPA em **React + TypeScript (Vite) [Decisão do grupo]**.
- P03 — **PostgreSQL** (um banco por serviço — *Database per Service*), **Redis** (refresh tokens, locks/TTL de reserva e cache do catálogo) **[Decisão do grupo]**.
- P04 — UI de mensageria: **Kafka UI** (provectus/kafbat) **[Decisão do grupo]**.
- P05 — Observabilidade com **OpenTelemetry + Prometheus + Grafana** (já vistos na disciplina).
- P06 — Pagamento e notificações são **simulados** (sem integração bancária ou provedores reais).
- P07 — Execução local/acadêmica; não há implantação em produção na nuvem.

2.10 Fora de escopo

- Integração com gateways bancários reais, emissão fiscal e split de pagamentos.
- Reembolso/cancelamento solicitado pelo comprador após o pagamento (apenas registro de requisito futuro).
- Revenda/transferência de ingressos entre usuários (mercado secundário).
- Aplicativo móvel nativo (a SPA será responsiva; a leitura de QR no operador usa a câmera via navegador).
- Fila virtual de espera completa estilo "sala de espera" (apenas limitação de taxa no gateway, podendo evoluir em release futura).
- Implantação em Kubernetes ou nuvem pública; alta disponibilidade multi-região.
- Envio real de SMS/push (simulado; e-mail via servidor de teste, ex.: MailHog/Mailpit).

2.11 Critérios de sucesso

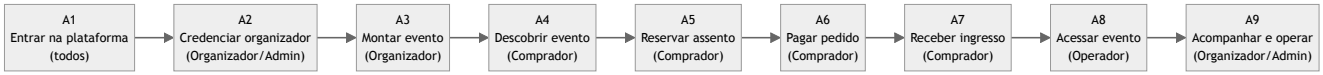
#	Critério	Como será medido
CS1	Zero overselling sob concorrência	Teste de carga com ≥ 1.000 compradores simultâneos disputando 100 assentos: exatamente 100 pedidos pagos, 0 assentos duplicados
CS2	Expiração em 10 min	Teste automatizado: reserva não paga volta ao estoque entre 10:00 e 10:05 min
CS3	Consistência banco ↔ Kafka	Teste de integração com falha simulada do broker: nenhum evento de pagamento perdido nem duplicado em efeito (Outbox + consumidor idempotente)
CS4	Fluxo ponta a ponta	Demonstração: comprador reserva, paga e recebe PDF com QR Code; operador valida o QR; organizador e admin veem o painel atualizado
CS5	Qualidade e reprodutibilidade	`docker compose up` sobe todo o ambiente; CI verde em todo PR para `develop`; cobertura de domínio ≥ 70%
CS6	Processo com IA rastreável	Specs versionadas no repositório para cada história implementada, com evidência de verificação
CS7	Entregas no prazo	Pacotes 1–4 entregues nas datas do cronograma (14/10, 04/11, 23/11, 02/12/2026)

3. Backlog com Épicos e Histórias de Usuários no Formato de Story Mapping

> Técnica: *User Story Mapping* (Jeff Patton). O **backbone** (eixo horizontal) é a jornada dos usuários; abaixo de cada atividade ficam as histórias, ordenadas verticalmente por prioridade e fatiadas em releases. Releases **[Decisão do grupo]**: **MVP** = Pacote 3 (checkpoint 23/11/2026) · **R2** = Pacote 4 (produto completo 02/12/2026) · **R3** = evolução futura (fora do prazo da disciplina, registrada para priorização).

3.1 Backbone (atividades do usuário, da esquerda para a direita)

A jornada principal é a do **Comprador**; à esquerda dela está a preparação feita pelo **Organizador** e pelo **Admin**, e à direita o acesso no dia (feito pelo **Operador**) e a análise (Organizador/Admin).



Atividade	A1 Entrar na plataforma	A2 Credenciar organizador	A3 Montar evento	A4 Descobrir evento	A5 Reservar assento	A6 Pagar pedido	A7 Receber ingresso	A8 Acessar evento	A9 Acompanhar e operar
Tarefas do usuário	Criar conta, logar, manter sessão, sair	Solicitar credenciamento, aprovar, gerir papéis, auditar	Criar evento, definir lotes, desenhar mapa, publicar	Buscar, filtrar, ver detalhes	Ver mapa, escolher, bloquear, acompanhar tempo	Revisar pedido, pagar, ver resultado	Ser notificado, baixar PDF, ver QR	Ler QR, validar, consultar	Ver vendas, métricas, saúde, pipeline

3.2 Épicos

ID	Épico	Módulo do enunciado	Atividades
EP1	Identidade, autenticação e sessão	Identidade e Governança (IAM)	A1
EP2	Governança: organizadores, papéis e auditoria	Identidade e Governança (IAM)	A2
EP3	Gestão de eventos, lotes e mapas de assentos	Inventário e Concorrência	A3
EP4	Catálogo de alta leitura	Inventário e Concorrência	A4
EP5	Reserva concorrente de assentos	Inventário e Concorrência	A5
EP6	Pedido, checkout e pagamento resiliente	Transações e Checkout	A6
EP7	Emissão de ingressos e notificações	Mensageria e Despacho	A7
EP8	Controle de acesso no evento	Mensageria e Despacho (+ IAM)	A8
EP9	Telemetria e painéis em tempo real	Análise e Operações	A9
EP10	DevOps: ambiente, CI/CD e qualidade	Análise e Operações	A9 (transversal)

3.3 Story Map (atividades × histórias, fatiado por release)

Cada célula lista os IDs das histórias (detalhadas em 3.4). De cima para baixo: maior → menor prioridade.

Release	A1 Entrar	A2 Credenciar	A3 Montar evento	A4 Descobrir	A5 Reservar	A6 Pagar	A7 Receber ingresso	A8 Acessar evento	A9 Acompanhar/operar
MVP (Pacote 3)	US01 Login Keycloak US02 Cadastro comprador US03 Refresh token	US05 Solicitar credenciamento US06 Aprovar organizador	US09 Criar evento US10 Criar lotes US11 Mapa de assentos US12 Publicar vendas	US14 Listar eventos US15 Detalhe do evento	US17 Ver mapa de assentos US18 Reservar assento US19 Expirar reserva em 10 min	US21 Criar pedido (FSM) US22 Pagar com Idempotency-Key US23 Outbox → Kafka	US25 Emitir ingresso PDF + QR US26 Meus ingressos	US29 Validar QR Code	US32 Docker Compose completo US33 CI no GitHub Actions

Release	A1 Entrar	A2 Credenciar	A3 Montar evento	A4 Descobrir	A5 Reservar	A6 Pagar	A7 Receber ingresso	A8 Acessar evento	A9 Acompanhar/operar
R2 (Pacote 4)	US04 Logout e revogação	US07 Gerir papéis (RBAC) US08 Trilha de auditoria	US13 Editar/encerrar lote	US16 Busca e filtros (cache)	US20 Disponibilidade em tempo real US20b Cancelar reserva	US24 Pagamento recusado / gateway fora	US27 Enviar comprovante por e-mail US28 Aviso em tempo real (WebSocket)	US30 Consultar ingresso/pedido	US34 Painel de vendas do organizador US35 Painel de telemetria (Admin) US36 Ingestão de eventos de navegação
R3 (futuro)	Login social	Convite de operadores por evento	Duplicar evento	Recomendações	Fila virtual de espera	Reembolso / cancelamento pós-pago	SMS / push reais; carteira digital	Validação offline no celular	US37 Alertas operacionais; testes de carga contínuos

3.4 Histórias de usuário detalhadas

Formato: *Como «papel», quero «ação», para «benefício»* + critérios de aceitação em Gherkin (Dado/Quando/Então). Prioridade MoSCoW (M = Must, S = Should, C = Could). Estimativa em pontos (Fibonacci) **[Decisão do grupo]**.

EP1 — Identidade, autenticação e sessão (IAM)

US01 — Login via Keycloak · M · MVP · 5 pts *Como* usuário de qualquer perfil, *quero* fazer login com e-mail e senha, *para* acessar as funções do meu papel com segurança.

- Dado um usuário cadastrado, quando informa credenciais válidas, então recebe um access token JWT (RS256, validade ≤ 5 min) e um refresh token, e é redirecionado à tela inicial do seu papel.
- Dado credenciais inválidas, quando tenta logar, então vê "E-mail ou senha inválidos" e a falha é registrada na auditoria.
- Dado um token ausente, expirado ou adulterado, quando chama uma rota protegida, então recebe HTTP 401.

US02 — Cadastro de comprador · M · MVP · 3 pts *Como* visitante, *quero* criar uma conta de comprador, *para* poder reservar e comprar ingressos.

- Dado e-mail não cadastrado e senha que atende à política (≥ 8 caracteres, letra e número), quando envio o cadastro, então a conta é criada no Keycloak com o papel **Cliente**.
- Dado e-mail já cadastrado, quando envio o cadastro, então vejo mensagem de conflito e nenhuma conta nova é criada.

US03 — Renovação de sessão com refresh token · M · MVP · 3 pts *Como* comprador, *quero* que minha sessão seja renovada automaticamente, *para* não perder minha reserva no meio do checkout.

- Dado um refresh token válido armazenado no Redis, quando o access token expira, então a SPA obtém um novo par de tokens sem intervenção do usuário.
- Dado um refresh token já utilizado (rotacionado), quando é reapresentado, então é rejeitado e todos os tokens daquela sessão são revogados.

US04 — Logout e revogação · S · R2 · 2 pts *Como* usuário, *quero* sair da plataforma, *para* impedir uso indevido da minha conta.

- Dado um usuário logado, quando clica em "Sair", então o refresh token é removido do Redis em ≤ 5 s e novas renovações falham.

EP2 — Governança: organizadores, papéis e auditoria (IAM)

US05 — Solicitar credenciamento de organizador · M · MVP · 3 pts *Como* produtora de eventos, *quero* solicitar cadastro como organizador informando razão social, CNPJ e contato, *para* poder vender ingressos na plataforma.

- Dado um CNPJ válido e ainda não cadastrado, quando envio a solicitação, então ela fica com status **Pendente** e o Admin é notificado.
- Dado um CNPJ inválido, quando envio, então vejo erro de validação.

US06 — Aprovar/recusar organizador · M · MVP · 3 pts *Como* Admin, *quero* aprovar ou recusar solicitações de organizadores, *para* garantir que só empresas legítimas criem eventos.

- Dado uma solicitação Pendente, quando aprovo, então a conta recebe o papel **Organizador** e a ação é registrada na auditoria.
- Dado uma solicitação recusada, quando o solicitante tenta criar evento, então recebe HTTP 403.

US07 — Gerir usuários e papéis (RBAC) · S · R2 · 5 pts *Como* Admin, *quero* atribuir e remover papéis (Cliente, Organizador, Operador, Admin) e bloquear usuários, *para* aplicar o menor privilégio.

- Dado um usuário, quando atribuo o papel Operador vinculado a um evento, então ele passa a ver a tela de validação apenas daquele evento.
- Dado um usuário bloqueado, quando tenta logar ou renovar sessão, então o acesso é negado.
- Toda alteração de papel gera registro de auditoria com autor, alvo, papel e data/hora.

US08 — Trilha de auditoria de credenciais · S · R2 · 5 pts *Como* Admin, *quero* consultar a trilha de auditoria de credenciais, *para* investigar acessos suspeitos.

- Dado eventos de login, falha, refresh, logout, revogação e mudança de papel, quando filtro por usuário e período, então vejo os registros com IP, data/hora (UTC) e resultado.
- Registros de auditoria não podem ser alterados nem excluídos pela API.

EP3 — Gestão de eventos, lotes e mapas de assentos (Inventário)

US09 — Criar evento · M · MVP · 3 pts *Como* organizador, *quero* cadastrar um evento com nome, descrição, local, data/hora e imagem, *para* divulgá-lo no catálogo.

- Dado um organizador aprovado, quando salvo o evento com campos obrigatórios, então ele é criado com status **Rascunho**, visível só para mim.
- Dado um organizador de outra empresa, quando tenta editar meu evento, então recebe HTTP 403.

US10 — Criar lotes · M · MVP · 3 pts *Como* organizador, *quero* definir lotes com preço, quantidade e janela de vendas, *para* controlar a precificação.

- Dado um evento em Rascunho, quando crio o lote "1º lote – Pista – R$ 150 – 500 un. – 01/11 a 10/11", então ele aparece na lista de lotes do evento.
- Dado uma janela de vendas que termina depois do evento, quando salvo, então vejo erro de validação.

US11 — Mapa de assentos por setor · M · MVP · 8 pts *Como* organizador, *quero* montar o mapa de setores, fileiras e assentos (numerados ou pista sem numeração), *para* que o comprador escolha seu lugar.

- Dado um setor numerado com 20 fileiras × 30 assentos, quando salvo, então 600 assentos são gerados com identificador único e status **Disponível**.
- Dado um setor de pista, quando defino capacidade 2.000, então o estoque é controlado por quantidade (sem numeração), com as mesmas garantias de atomicidade.

US12 — Publicar e encerrar vendas · M · MVP · 2 pts *Como* organizador, *quero* publicar o evento, *para* que ele apareça no catálogo e as vendas abram na data configurada.

- Dado um evento com ao menos um lote e mapa, quando publico, então ele aparece no catálogo e o cache do catálogo é invalidado em ≤ 5 s.
- Dado um evento sem lote, quando tento publicar, então a publicação é bloqueada com mensagem explicativa.

US13 — Editar ou encerrar lote · S · R2 · 3 pts *Como* organizador, *quero* encerrar um lote antes do fim, *para* abrir o próximo lote.

- Dado um lote com reservas ativas, quando o encerro, então novas reservas são bloqueadas, mas as reservas ativas podem ser pagas até expirarem.

EP4 — Catálogo de alta leitura (Inventário)

US14 — Listar eventos · M · MVP · 3 pts *Como* comprador, *quero* ver a lista de eventos publicados, *para* escolher o que comprar.

- Dado eventos publicados, quando acesso a home, então vejo cartões com nome, data, cidade, menor preço e indicador "Esgotado" quando aplicável.
- A listagem é servida por cache (RNF09: p95 ≤ 200 ms a 2.000 req/s).

US15 — Detalhe do evento · M · MVP · 2 pts *Como* comprador, *quero* ver os detalhes do evento e os lotes à venda, *para* decidir o setor e o preço.

- Dado um evento com vendas abertas, quando abro o detalhe, então vejo descrição, local, lotes ativos com preço e botão "Escolher assentos".
- Dado um evento com vendas ainda não abertas, então vejo a contagem regressiva para a abertura e o botão desabilitado.

US16 — Busca e filtros · S · R2 · 3 pts *Como* comprador, *quero* buscar por nome e filtrar por cidade, data e categoria, *para* encontrar eventos rapidamente.

- Dado o termo "festival" e o filtro "Porto Alegre", quando busco, então vejo apenas eventos publicados que atendem aos dois critérios.

EP5 — Reserva concorrente de assentos (Inventário)

US17 — Ver mapa de assentos · M · MVP · 5 pts *Como* comprador, *quero* ver o mapa do setor com assentos disponíveis, reservados e vendidos, *para* escolher meu lugar.

- Dado um setor, quando abro o mapa, então cada assento aparece com cor/legenda por status e preço do lote.

US18 — Reservar assento atomicamente · M · MVP · 8 pts *Como* comprador, *quero* reservar até 6 assentos de uma vez, *para* garantir meus lugares enquanto pago.

- Dado assentos Disponíveis, quando confirmo a seleção, então todos são bloqueados atomicamente (tudo-ou-nada) por 10 min, um pedido é criado e eu vejo a contagem regressiva.
- Dado que outro comprador reservou um dos assentos um instante antes, quando confirmo, então **nenhum** assento é bloqueado e vejo "Assento indisponível, escolha outro".
- Dado 1.000 compradores disputando o mesmo assento ao mesmo tempo, então exatamente 1 reserva é concedida (**zero overselling**, RNF01).
- Dado que excedo o limite de 6 assentos por pedido, então a reserva é recusada.

US19 — Expirar reserva em 10 minutos · M · MVP · 5 pts *Como* organizador, *quero* que reservas não pagas expirem automaticamente em 10 minutos, *para* que os assentos voltem à venda.

- Dado uma reserva criada às 20:00:00 e não paga, quando chega 20:10:00, então em até 5 s o pedido vai para **Expirado**, os assentos voltam a **Disponível** atomicamente e o evento `ReservaExpirada` é publicado no Kafka.
- Dado que o pagamento é confirmado às 20:09:59, então a reserva **não** expira (a transição para Pago prevalece e a expiração tardia é ignorada de forma idempotente).
- O comprador vê na tela "Sua reserva expirou" e pode escolher novos assentos.

US20 — Disponibilidade em tempo real · S · R2 · 5 pts *Como* comprador, *quero* que o mapa de assentos se atualize sozinho, *para* não tentar reservar lugares já ocupados.

- Dado dois compradores com o mesmo mapa aberto, quando um reserva um assento, então ele aparece como Reservado na tela do outro em ≤ 2 s (WebSocket).

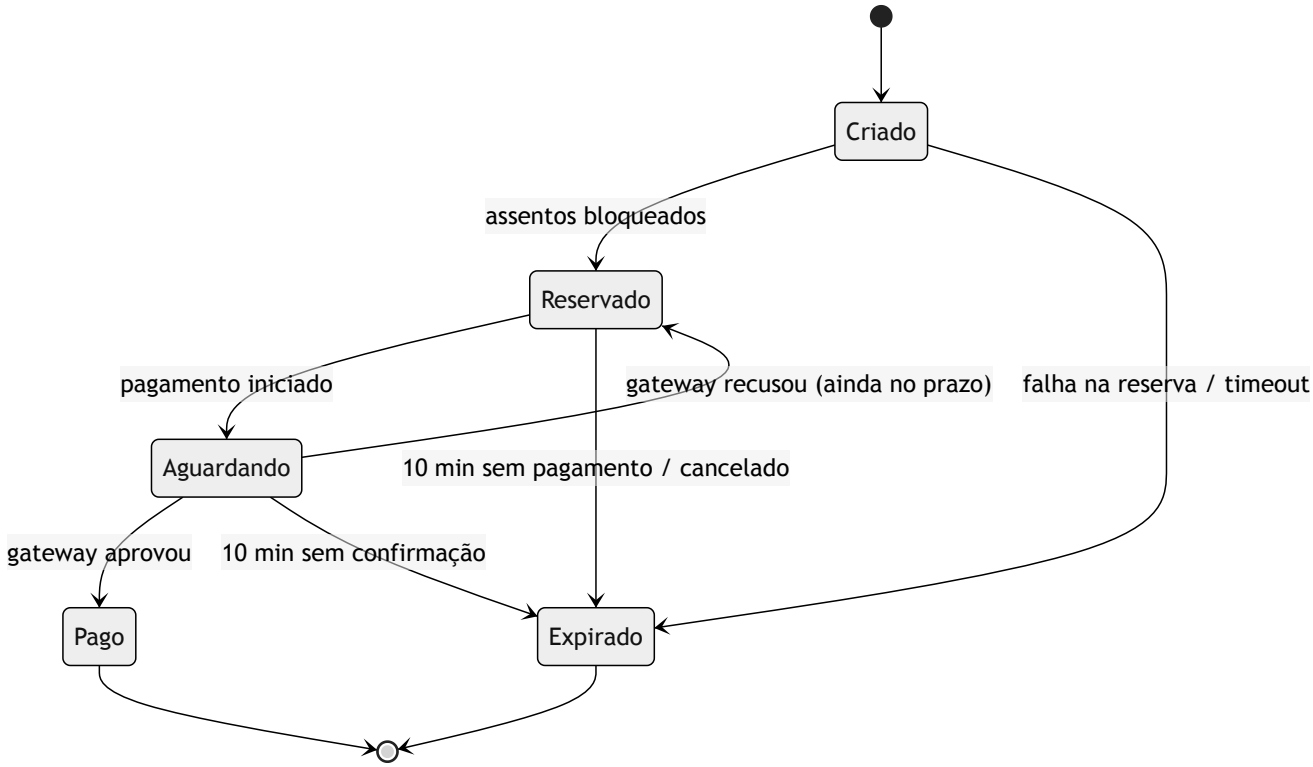
US20b — Cancelar reserva · C · R2 · 2 pts *Como* comprador, *quero* desistir da reserva antes de pagar, *para* liberar os assentos para outras pessoas.

- Dado uma reserva ativa, quando clico em "Cancelar reserva", então o pedido vai para **Expirado** (motivo: cancelado pelo usuário) e os assentos são devolvidos atomicamente.

EP6 — Pedido, checkout e pagamento resiliente (Checkout)

US21 — Pedido com máquina de estados · M · MVP · 8 pts *Como* comprador, *quero* acompanhar o status do meu pedido, *para* saber se a compra foi concluída.

- O pedido segue a FSM: **Criado** (itens escolhidos) → **Reservado** (assentos bloqueados) → **Aguardando** (pagamento enviado ao gateway) → **Pago**; de Criado, Reservado ou Aguardando pode ir para **Expirado**.
- Dado um pedido Pago, quando chega um evento de expiração, então ele permanece Pago (transição inválida rejeitada e registrada).
- Dado um pedido Expirado, quando chega confirmação de pagamento tardia, então o pedido não volta para Pago e é gerado evento para estorno simulado.



US22 — Pagar com Idempotency-Key · M · MVP · 5 pts *Como* comprador, *quero* que um clique duplo ou uma reconexão não me cobre duas vezes, *para* pagar com segurança.

- Dado uma requisição `POST /pedidos/{id}/pagamento` com cabeçalho `Idempotency-Key: <uuid>`, quando ela é reenviada com a mesma chave, então a resposta original é devolvida e o gateway é chamado apenas uma vez.
- Dado uma requisição sem `Idempotency-Key`, então a API responde HTTP 400.
- Dado a mesma chave com corpo diferente, então a API responde HTTP 422.

US23 — Publicação confiável via Transactional Outbox · M · MVP · 8 pts *Como* time da plataforma, *quero* que a mudança de estado do pedido e o evento correspondente sejam gravados na mesma transação, *para* nunca emitir ingresso sem pagamento nem perder pagamento sem ingresso (dual-write).

- Dado um pagamento aprovado, quando o pedido vai para Pago, então na mesma transação ACID é inserido na tabela `outbox` o evento `PagamentoConfirmado`.
- Dado que o Kafka está indisponível, quando volta, então o *relay* publica todos os eventos pendentes, em ordem por pedido.
- Dado uma falha após a publicação e antes de marcar o registro como enviado, então o evento pode ser republicado, e os consumidores o tratam de forma idempotente (US25).

US24 — Pagamento recusado ou gateway indisponível · S · R2 · 5 pts *Como* comprador, *quero* ser avisado se o pagamento falhar, *para* tentar de novo dentro do prazo da reserva.

- Dado que o simulador recusa o pagamento, então o pedido volta para Reservado (mantendo o prazo original de expiração) e vejo "Pagamento recusado".
- Dado que o gateway não responde em 2 s, então o Circuit Breaker atua, vejo "Tente novamente em instantes" e nenhuma thread fica presa.

EP7 — Emissão de ingressos e notificações (Mensageria e Despacho)

US25 — Emitir ingresso em PDF com QR Code · M · MVP · 8 pts *Como* comprador, *quero* receber um ingresso em PDF com QR Code, *para* entrar no evento.

- Dado um evento `PagamentoConfirmado` no Kafka, quando consumido, então é gerado um ingresso por assento com PDF contendo nome do evento, setor/assento, titular, **carimbo de tempo** da emissão e **QR Code assinado**.
- Dado que o mesmo evento é entregue duas vezes, então apenas um ingresso por assento existe (consumo idempotente por `eventId`).
- Dado que a geração falha 3 vezes, então a mensagem vai para a DLQ e um alerta é gerado.

US26 — Meus ingressos · M · MVP · 3 pts *Como* comprador, *quero* ver e baixar meus ingressos, *para* tê-los à mão no dia do evento.

- Dado ingressos emitidos, quando abro "Meus ingressos", então vejo a lista com QR Code e botão "Baixar PDF".
- Dado outro usuário, quando tenta acessar meu ingresso pela URL, então recebe HTTP 403.

US27 — Comprovante multicanal · S · R2 · 3 pts *Como* comprador, *quero* receber o comprovante e o PDF por e-mail (e, futuramente, SMS/push), *para* ter uma cópia fora da plataforma.

- Dado um ingresso emitido, então um e-mail com o PDF anexado é enviado (servidor de teste) em ≤ 30 s.
- Falha de envio é reprocessada com *backoff* sem gerar ingresso duplicado.

US28 — Atualização de tela em tempo real · S · R2 · 5 pts *Como* comprador, *quero* que a tela do pedido mude sozinha para "Pago" e "Ingresso emitido", *para* não precisar recarregar.

- Dado a tela de confirmação aberta, quando o ingresso é emitido, então recebo a notificação via WebSocket em ≤ 2 s e o botão "Baixar ingresso" aparece.

EP8 — Controle de acesso no evento

US29 — Validar QR Code na entrada · M · MVP · 5 pts *Como* operador, *quero* ler o QR Code do ingresso, *para* liberar ou negar a entrada.

- Dado um ingresso válido e não utilizado, quando leio o QR, então vejo **"Acesso liberado"** (verde) e o ingresso passa a **Utilizado** com data/hora.
- Dado um ingresso já utilizado, então vejo **"Já utilizado em «data/hora»"** (amarelo).
- Dado um QR com assinatura inválida ou de outro evento, então vejo **"Ingresso inválido"** (vermelho).
- Dado um operador não vinculado ao evento, então a validação é negada (HTTP 403).

US30 — Consultar ingresso/pedido · S · R2 · 2 pts *Como* operador, *quero* buscar um ingresso por código ou CPF do titular, *para* resolver problemas na portaria.

- Dado um código válido, quando busco, então vejo status do pedido e do ingresso, sem dados de pagamento.

EP9 — Telemetria e painéis em tempo real (Análise e Operações)

US34 — Painel de vendas do organizador · S · R2 · 5 pts *Como* organizador, *quero* ver vendas por lote, ocupação e receita em tempo real, *para* decidir quando abrir o próximo lote.

- Dado vendas acontecendo, quando abro o painel, então os números são atualizados com atraso ≤ 10 s e vejo apenas meus eventos.

US35 — Painel de telemetria da plataforma · S · R2 · 5 pts *Como* Admin, *quero* um painel com requisições/s, latência p95, erros, reservas ativas/expiradas e *consumer lag* do Kafka, *para* agir rápido durante a abertura de vendas.

- Dado o ambiente em execução, quando abro o painel (Grafana embutido ou tela da SPA), então vejo as métricas atualizadas a cada ≤ 15 s.

US36 — Ingestão de eventos de navegação e compra · S · R2 · 5 pts *Como* organizador, *quero* que visualizações de evento, início de checkout e compras sejam registradas, *para* medir a conversão do funil.

- Dado eventos de navegação enviados pela SPA e eventos de domínio no Kafka, quando agregados, então o funil (visualizou → reservou → pagou) é exibido por evento.

US37 — Alertas operacionais · C · R3 · 3 pts *Como* Admin, *quero* receber alertas quando a taxa de erro passar de 1% ou o lag passar de 10.000 mensagens, *para* reagir antes de afetar os compradores.

EP10 — DevOps: ambiente, CI/CD e qualidade (Análise e Operações)

US32 — Ambiente completo em docker-compose · M · MVP · 5 pts *Como* desenvolvedor do grupo, *quero* subir todo o ambiente com um comando, *para* desenvolver e demonstrar sem configuração manual.

- Dado uma máquina com Docker, quando executo `docker compose up -d`, então sobem SPA, gateway, microsserviços, PostgreSQL, Redis, Kafka, Kafka UI, Keycloak (com realm e papéis importados), Prometheus, Grafana e servidor de e-mail de teste, todos saudáveis em ≤ 5 min.

US33 — Pipeline de CI no GitHub Actions · M · MVP · 5 pts *Como* desenvolvedor do grupo, *quero* que todo PR para `develop` rode lint, testes e build, *para* só integrar código verificado.

- Dado um PR aberto para `develop`, então o pipeline executa lint, testes unitários (com *gate* de cobertura de domínio ≥ 70%), testes de integração com Testcontainers e build das imagens.
- Dado um pipeline vermelho, então o merge é bloqueado pela proteção de branch.

3.5 Resumo do backlog

Release	Histórias	Pontos
MVP	US01–03, US05–06, US09–12, US14–15, US17–19, US21–23, US25–26, US29, US32–33	103
R2	US04, US07–08, US13, US16, US20, US20b, US24, US27–28, US30, US34–36	55
R3	US37 + itens registrados no mapa	—

4. Requisitos Não-Funcionais

> Categorias segundo a ISO/IEC 25010. Cada RNF tem métrica/alvo verificável e forma de verificação. Módulos: **IAM** (Identidade e Governança) · **INV** (Inventário e Concorrência) · **CHK** (Transações e Checkout) · **TKT** (Emissão de Ingressos e Notificações) · **TEL** (Telemetria e DevOps). Valores numéricos de desempenho são metas definidas pelo grupo para o ambiente local de referência (docker-compose em um notebook com 8 vCPU / 16 GB) **[Decisão do grupo]**.

4.1 Consistência, concorrência e integridade transacional

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF01	Confiabilidade — Integridade	**Zero overselling**: um assento só pode estar associado a uma reserva ativa ou a um pedido pago por vez.	0 assentos com mais de um pedido Pago/Reservado em qualquer teste; restrição `UNIQUE (evento_id, assento_id)` em ingressos válidos	Teste de carga (k6/Gatling) com 1.000 usuários concorrentes disputando 100 assentos + consulta SQL de duplicidades; teste de integração concorrente (Testcontainers, 50 threads no mesmo assento → exatamente 1 sucesso)	INV, CHK
RNF02	Confiabilidade — Integridade	**Alocação e devolução atômicas** de assentos (operação tudo-ou-nada, inclusive para múltiplos assentos no mesmo pedido).	0 estados parciais (ex.: 2 de 3 assentos bloqueados) após falhas injetadas	Teste de integração com falha no meio da operação; verificação de invariantes no banco	INV
RNF03	Confiabilidade — Tempo	**Expiração estrita de reserva em 10 minutos**: reserva não paga é expirada e o assento volta ao estoque.	Assento disponível novamente entre 600 s e 605 s após a reserva (tolerância ≤ 5 s); pedido vai para `Expirado`; evento `ReservaExpirada` publicado	Teste com relógio controlável (`Clock` injetado) + teste de integração com TTL do Redis e job de varredura; métrica `reservas_expiradas_atraso_segundos` (p99 ≤ 5 s)	INV, CHK
RNF04	Confiabilidade — Correção	**FSM do pedido**: somente transições válidas Criado→Reservado→Aguardando→Pago e {Criado, Reservado, Aguardando} →Expirado.	100% das transições inválidas rejeitadas; cobertura de ramos da FSM ≥ 90%	Testes unitários parametrizados (todas as combinações estado × evento) + teste de mutação (PIT) com *mutation score* ≥ 70% no pacote de domínio	CHK
RNF05	Confiabilidade — Idempotência HTTP	Endpoints que criam/alteram pedidos e pagamentos exigem cabeçalho **`Idempotency-Key`**; repetições retornam a mesma resposta sem novo efeito.	0 pedidos/cobranças duplicadas em 100 reenvios da mesma chave; chave armazenada por ≥ 24 h; requisição sem chave → HTTP 400	Teste de integração enviando a mesma requisição N vezes (incluindo concorrentemente)	CHK
RNF06	Confiabilidade — Dual-write	**Transactional Outbox**: alteração de estado e registro do evento na mesma transação ACID do PostgreSQL; publicação no Kafka por *relay*/worker.	0 eventos perdidos; atraso Outbox→Kafka p95 ≤ 2 s em operação normal	Teste de integração (Testcontainers) derrubando o Kafka durante o checkout e religando: todos os eventos são publicados depois; nenhuma escrita direta no Kafka dentro do caso de uso (verificado por ArchUnit)	CHK, INV
RNF07	Confiabilidade — Idempotência no consumo	Consumidores Kafka toleram entrega *at-least-once*: mensagens duplicadas não geram efeito duplicado.	0 ingressos duplicados para o mesmo pedido após reprocessar o tópico inteiro	Teste reenviando o mesmo evento 10× (tabela de mensagens processadas / chave de deduplicação por `eventId`)	TKT, TEL, INV
RNF08	Confiabilidade — Mensagens inválidas	Mensagens que falham repetidamente vão para **DLQ** sem bloquear a partição.	Após 3 tentativas com *backoff*, mensagem em `<topico>.dlq`; consumo das demais continua	Teste de integração com mensagem envenenada	TKT, TEL

4.2 Desempenho e escalabilidade (flash crowd)

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF09	Eficiência — Tempo de resposta	Catálogo de eventos de **alta leitura** servido por cache.	p95 ≤ 200 ms a 2.000 req/s de leitura; taxa de acerto do cache ≥ 90%	Teste de carga k6; métricas Prometheus (`http_server_requests_seconds`, *hit ratio*)	INV

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF10	Eficiência — Tempo de resposta	Reserva de assento sob pico.	p95 ≤ 500 ms, p99 ≤ 1 s com 1.000 usuários virtuais simultâneos; taxa de erro 5xx < 1%	Teste de carga de abertura de vendas (rampa de 0→1.000 VUs em 10 s)	INV, CHK
RNF11	Eficiência — Capacidade	**Degradação controlada**: acima da capacidade, o gateway aplica *rate limiting* por usuário/IP em vez de derrubar serviços.	Respostas HTTP 429 com `Retry-After`; nenhum serviço reinicia por OOM ou esgotamento de pool durante o teste	Teste de carga 2× acima da meta; inspeção de *restarts* dos containers e do pool HikariCP	Todos (API Gateway)
RNF12	Eficiência — Assincronismo	Etapas lentas (gerar PDF, enviar e-mail) não ficam no caminho síncrono do pagamento.	Confirmação de pagamento ao comprador p95 ≤ 1 s; ingresso emitido p95 ≤ 10 s após `PagamentoConfirmado`	Métrica de latência ponta a ponta com *trace* OpenTelemetry	CHK, TKT
RNF13	Escalabilidade	Serviços sem estado em memória (exceto cache), escaláveis horizontalmente; consumidores escaláveis por partição.	Escalar INV e TKT para 2 réplicas no compose sem alteração de código; tópicos com ≥ 3 partições	`docker compose up --scale` + teste de carga repetido	INV, TKT, CHK

4.3 Disponibilidade e resiliência

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF14	Confiabilidade — Disponibilidade	Disponibilidade da jornada de compra durante janela de vendas.	≥ 99,5% das requisições com sucesso durante o teste de carga de 30 min	Relatório k6 + painel Grafana	Todos
RNF15	Confiabilidade — Tolerância a falhas	Chamadas síncronas entre serviços e ao gateway de pagamento com **timeout, retry com backoff e Circuit Breaker** (Resilience4j).	Timeout ≤ 2 s; circuito abre após 50% de falhas em 20 chamadas; falha do gateway não bloqueia threads do checkout	Teste com simulador de pagamento configurado para falhar/atrasar	CHK
RNF16	Confiabilidade — Recuperabilidade	Reinício de qualquer serviço não perde pedidos nem eventos.	Após `docker restart` de CHK ou TKT durante carga: 0 pedidos órfãos; consumidores retomam do último *offset* confirmado	Teste de caos manual roteirizado	CHK, TKT
RNF17	Confiabilidade — Health	Todos os serviços expõem *health checks* (liveness/readiness).	`/actuator/health` em todos; compose usa `healthcheck` e `depends_on: condition: service_healthy`	Inspeção do compose + CI sobe o ambiente e checa saúde	Todos

4.4 Segurança

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF18	Segurança — Autenticidade	Autenticação **stateless via JWT** emitido pelo **Keycloak**, assinado (RS256) e validado no gateway e em cada serviço.	Access token com validade ≤ 5 min; 100% das rotas não públicas rejeitam token ausente/expirado/adulterado (HTTP 401)	Testes de integração de segurança por rota	IAM, todos
RNF19	Segurança — Sessão	**Refresh tokens armazenados em chave-valor (Redis)** com rotação e revogação.	Refresh token com validade ≤ 8 h; refresh token reutilizado após rotação é rejeitado; logout/revogação efetivo em ≤ 5 s	Teste de integração com Redis (Testcontainers)	IAM
RNF20	Segurança — Autorização	**RBAC** com papéis Cliente, Organizador, Operador, Admin; princípio do menor privilégio; organizador só acessa os próprios eventos.	100% dos endpoints com papel declarado; matriz papel × endpoint coberta por testes (acesso indevido → HTTP 403)	Testes de autorização gerados a partir da matriz; ArchUnit garante anotação de segurança em todo controller	IAM, todos
RNF21	Segurança — Auditoria	**Auditoria de credenciais**: login, falha de login, refresh, logout, revogação, mudança de papel, aprovação de organizador.	100% desses eventos registrados com usuário, IP, data/hora (UTC) e resultado; registros imutáveis (*append-only*); retenção ≥ 90 dias	Teste de integração verificando registro para cada tipo de evento	IAM
RNF22	Segurança — Integridade do ingresso	QR Code contém identificador e **assinatura digital** (HMAC/assinatura assimétrica) verificável; ingresso só pode ser usado uma vez.	0 ingressos falsificados aceitos em teste; segunda leitura do mesmo QR → "já utilizado"	Testes unitários de assinatura + teste de validação dupla	TKT
RNF23	Segurança — Proteção de dados	Comunicação via HTTPS no gateway; senhas apenas no Keycloak (hash); dados pessoais mínimos (LGPD); **nenhum segredo no repositório**.	0 segredos detectados pelo *secret scanning* (gitleaks) no CI; segredos via variáveis de ambiente/`.env` fora do Git	Etapa de CI com gitleaks; revisão de PR	Todos

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF24	Segurança — Vulnerabilidades	Dependências e imagens verificadas.	0 vulnerabilidades críticas abertas (OWASP Dependency-Check/Trivy) no merge para `main`	Etapa de CI	Todos

4.5 Observabilidade e telemetria

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF25	Manutenibilidade — Observabilidade	Métricas técnicas (RED: taxa, erros, duração) e de negócio (vendas/min, reservas ativas, expiradas, conversão) expostas para **Prometheus** e exibidas no **Grafana**.	100% dos serviços com `/actuator/prometheus`; painéis atualizados com atraso ≤ 15 s	Inspeção dos painéis provisionados no compose	TEL, todos
RNF26	Manutenibilidade — Rastreamento	**Rastreamento distribuído** (OpenTelemetry) propagando *trace id* por HTTP e cabeçalhos Kafka.	100% das requisições de checkout com trace ponta a ponta (gateway → CHK → Kafka → TKT)	Visualização no Jaeger/Tempo durante a demo	Todos
RNF27	Manutenibilidade — Logs	Logs estruturados em JSON com `traceId`, `userId` (quando aplicável) e sem dados sensíveis (tokens, cartão).	0 tokens/senhas em logs (teste de regex no CI sobre logs dos testes de integração)	Teste automatizado + revisão	Todos
RNF28	Funcionalidade — Telemetria de negócio	Ingestão e agregação contínua de eventos de navegação e compra a partir dos tópicos Kafka.	Atraso de agregação ≤ 10 s; *consumer lag* monitorado com alerta se > 10.000 mensagens por 1 min	Painel de lag no Grafana + teste de integração do agregador	TEL
RNF29	Usabilidade — Tempo real	Atualização de tela em tempo real via **WebSocket** (disponibilidade de assentos, status do pedido, ingresso emitido).	Mudança refletida na tela em ≤ 2 s (p95); reconexão automática em ≤ 5 s	Teste E2E (Playwright) com dois navegadores	TKT, INV

4.6 Testabilidade, manutenibilidade e qualidade de código

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF30	Manutenibilidade — Testabilidade	Testes unitários das regras de domínio e máquinas de estado.	**≥ 70% de cobertura de linhas no domínio** (JaCoCo), ≥ 90% de ramos na FSM do pedido; CI falha abaixo do limite	Relatório JaCoCo com *quality gate* no CI	Todos
RNF31	Manutenibilidade — Testabilidade	Testes de integração com **Testcontainers reais** de PostgreSQL, Kafka e Redis (sem mocks de infraestrutura).	Todo repositório, produtor/consumidor Kafka e uso de Redis com ≥ 1 teste de integração	Execução no CI (`mvn verify`)	Todos
RNF32	Manutenibilidade — Eficácia dos testes	Teste de mutação no domínio.	*Mutation score* ≥ 70% (PIT) nos pacotes de domínio de INV e CHK	Relatório PIT no CI (job semanal ou em PR para `main`)	INV, CHK
RNF33	Manutenibilidade — Testes de carga (bônus)	Cenários de carga versionados.	Scripts k6 para catálogo, reserva e checkout no repositório; resultados anexados aos Pacotes 3/4	Execução documentada	INV, CHK
RNF34	Manutenibilidade — Modularidade	Arquitetura em camadas/hexagonal por serviço; domínio sem dependência de framework; *Database per Service*.	0 violações de regras ArchUnit (domínio não importa Spring/JPA/Kafka; serviço não acessa banco de outro)	ArchUnit executado nos testes	Todos
RNF35	Manutenibilidade — Analisabilidade	Padrões de código e análise estática.	Checkstyle/Spotless e ESLint/Prettier sem erros; SonarLint/Sonar sem *bugs* bloqueantes	Etapa de lint no CI	Todos, SPA
RNF36	Manutenibilidade — Contratos	APIs documentadas em **OpenAPI 3** e eventos Kafka com esquema versionado.	100% dos endpoints públicos no OpenAPI; eventos com campo `schemaVersion` e mudanças retrocompatíveis	Geração do OpenAPI no build; testes de contrato dos eventos	Todos

4.7 Portabilidade, implantação e CI/CD

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF37	Portabilidade — Instalabilidade	Ambiente completo orquestrado via **`docker-compose.yml`**: serviços back-end, front-end, PostgreSQL, Redis, Kafka, **Kafka UI**, Keycloak, Prometheus, Grafana, servidor de e-mail de teste.	`docker compose up -d` sobe tudo saudável em ≤ 5 min numa máquina limpa, sem passos manuais além de copiar `.env.example`	Teste no CI (job de *smoke*) e em máquina de outro integrante	TEL, todos
RNF38	Portabilidade — Adaptabilidade	Configuração externalizada (variáveis de ambiente / Spring Cloud Config).	0 URLs/credenciais *hard-coded*; mesmo artefato roda em dev e CI	Revisão + ArchUnit/grep no CI	Todos
RNF39	Manutenibilidade — CI	**Pipeline de CI no GitHub Actions** executando lint, testes (unit + integração) e build a cada Pull Request e merge em `develop`.	100% dos PRs para `develop` com CI verde antes do merge (regra de proteção de branch); duração do pipeline ≤ 15 min	Histórico do GitHub Actions; configuração de *branch protection*	TEL

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF40	Manutenibilidade — CD	Build e publicação de imagens Docker versionadas ao fazer merge em `main`.	Imagens com tag do commit/semver publicadas no GHCR	Histórico do pipeline	TEL

4.8 Usabilidade e acessibilidade

ID	Categoria	Requisito	Métrica / alvo	Como verificar	Módulos
RNF41	Usabilidade — Operabilidade	Compra concluída em poucos passos.	Do evento ao pagamento em ≤ 4 telas / ≤ 6 cliques	Revisão dos mockups e teste E2E	SPA
RNF42	Usabilidade — Feedback	Contagem regressiva visível da reserva e mensagens claras para "assento indisponível", "reserva expirada" e "pagamento recusado".	100% desses estados com mensagem específica na interface	Teste E2E	SPA
RNF43	Usabilidade — Acessibilidade	SPA responsiva e acessível.	WCAG 2.1 nível AA nas telas do comprador (Lighthouse Accessibility ≥ 90); funciona em telas ≥ 360 px	Lighthouse no CI do front	SPA
RNF44	Usabilidade — Idioma	Interface em português (pt-BR), datas no fuso America/Sao_Paulo e valores em R$.	100% das telas	Revisão	SPA

4.9 Rastreabilidade RNF × módulos

Módulo	RNFs principais
IAM	RNF18, RNF19, RNF20, RNF21, RNF23
Inventário e Concorrência	RNF01, RNF02, RNF03, RNF09, RNF10, RNF13, RNF29
Transações e Checkout	RNF01, RNF04, RNF05, RNF06, RNF12, RNF15, RNF16
Emissão de Ingressos e Notificações	RNF07, RNF08, RNF12, RNF22, RNF29
Telemetria e DevOps	RNF25, RNF26, RNF28, RNF37, RNF39, RNF40
Transversais	RNF11, RNF14, RNF17, RNF24, RNF27, RNF30–RNF36, RNF38, RNF41–RNF44

5. Mockups (medium res) e Mapa de Navegação do Front-end

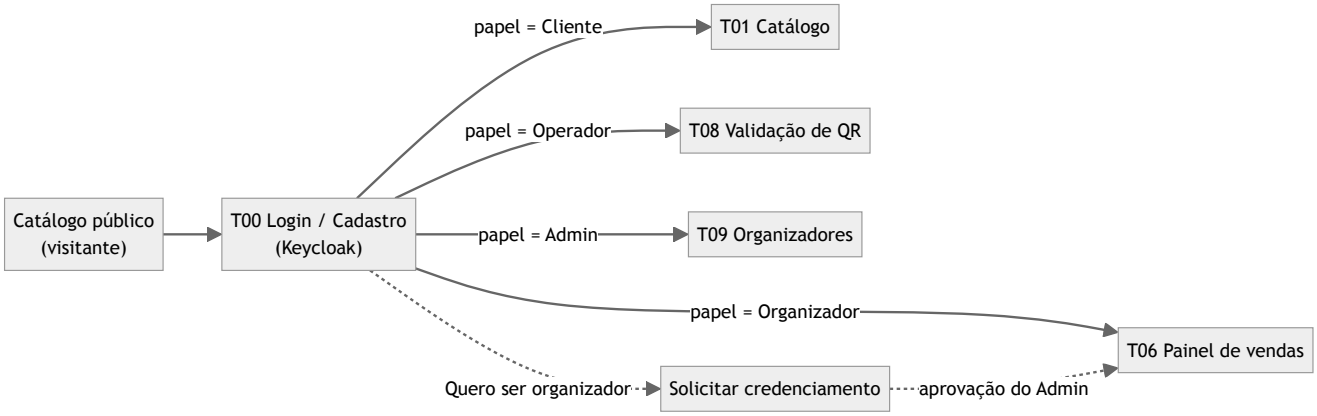
5.1 Fidelidade escolhida e ferramenta

- **Fidelidade: média (*medium-fi*)** — wireframes em tons de cinza com uma única cor de destaque (roxo, ações primárias), dados fictícios realistas e estados de erro/borda anotados. Não representam a identidade visual final.
- **Ferramenta:** HTML + CSS simples (`mockups/*.html` + `mockups/wireframe.css`), capturados em PNG com Chrome *headless* (`mockups/img/`). A escolha permite versionar os mockups no Git, revisá-los em PR e gerá-los/ajustá-los com apoio do agente de IA (ver item 1) **[Decisão do grupo]**.
- **Front-end previsto:** SPA em React + TypeScript, responsiva, com rotas protegidas por papel (RBAC lido do JWT do Keycloak) e canal WebSocket para atualizações em tempo real.
- **Notas amarelas** nos mockups indicam comportamento não visível em imagem estática (ex.: atualização via WebSocket).

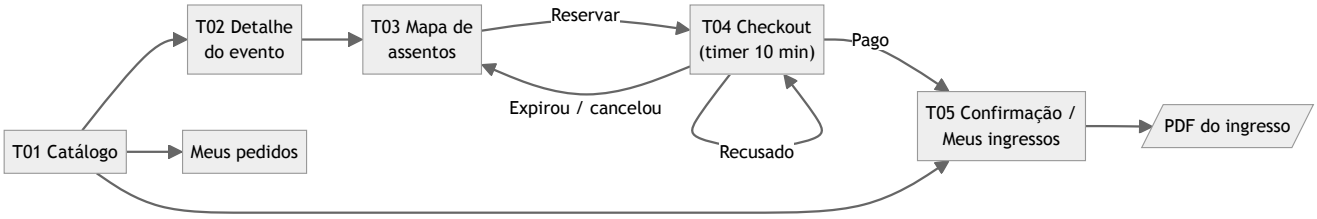
5.2 Mapa de navegação

5.2.1 Visão geral por perfil

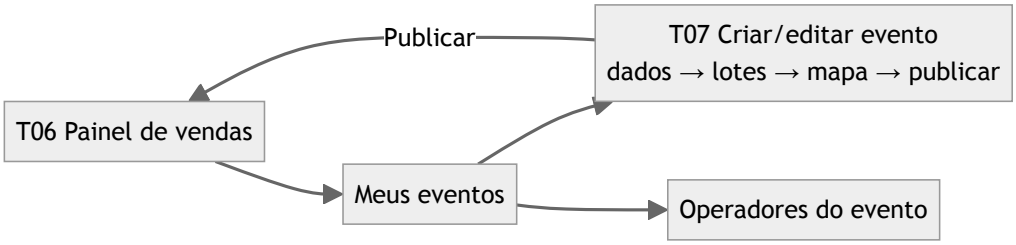
Após o login (Keycloak), a SPA redireciona para a tela inicial do papel do usuário. Um usuário com mais de um papel pode alternar pelo menu do perfil.



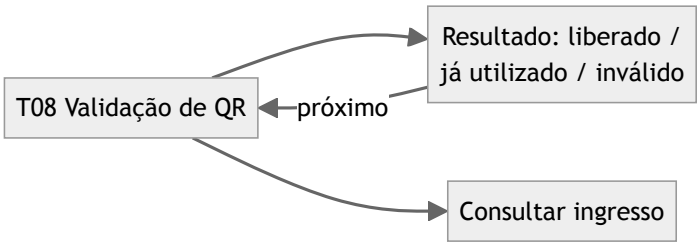
Comprador



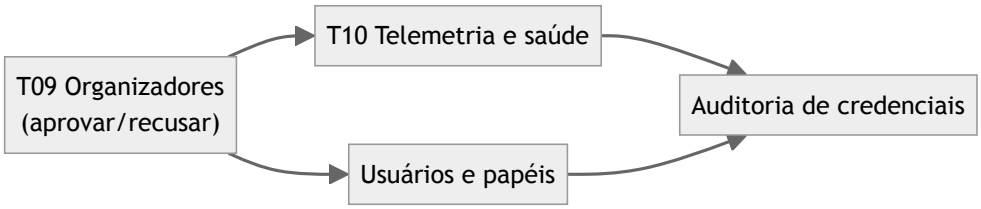
Organizador



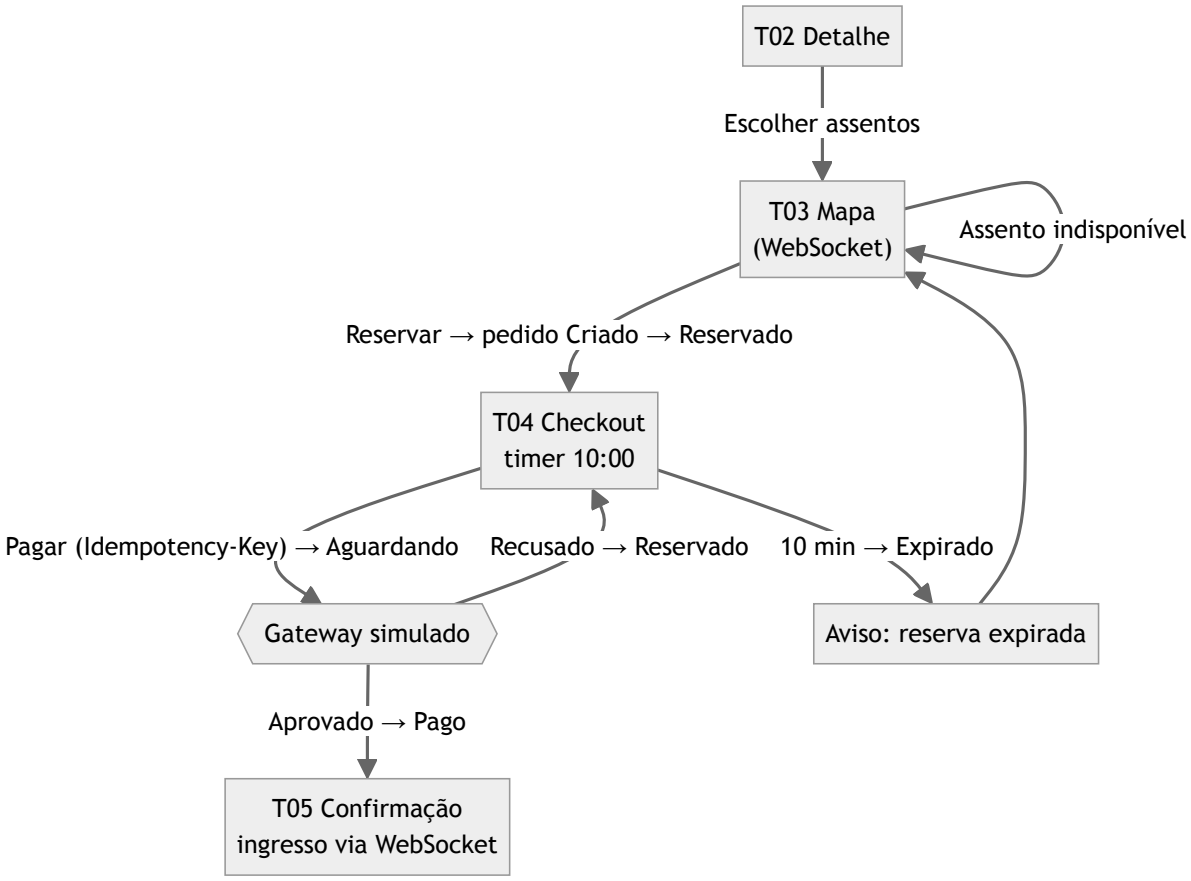
Operador



Admin



5.2.2 Fluxo crítico de compra (com estados do pedido)



5.2.3 Rotas da SPA e papéis

Tela	Rota	Papel exigido	Histórias
T00 Login / Cadastro	`/entrar`, `/cadastro`	público	US01, US02, US05
T01 Catálogo	`/` , `/eventos?busca=`	público	US14, US16
T02 Detalhe do evento	`/eventos/:id`	público	US15
T03 Mapa de assentos	`/eventos/:id/assentos`	Cliente	US17, US18, US20
T04 Checkout	`/pedidos/:id/pagamento`	Cliente	US19, US21, US22, US24

Tela	Rota	Papel exigido	Histórias
T05 Meus ingressos	`/meus-ingressos`	Cliente	US25, US26, US28
Meus pedidos	`/meus-pedidos`	Cliente	US21
T06 Painel de vendas	`/organizador`	Organizador	US34
T07 Criar/editar evento	`/organizador/eventos/:id/editar`	Organizador	US09–US13
T08 Validação de QR	`/operador/validar`	Operador	US29, US30
T09 Organizadores / usuários	`/admin/organizadores`, `/admin/usuarios`	Admin	US06, US07
T10 Telemetria / auditoria	`/admin/telemetria`, `/admin/auditoria`	Admin	US08, US35

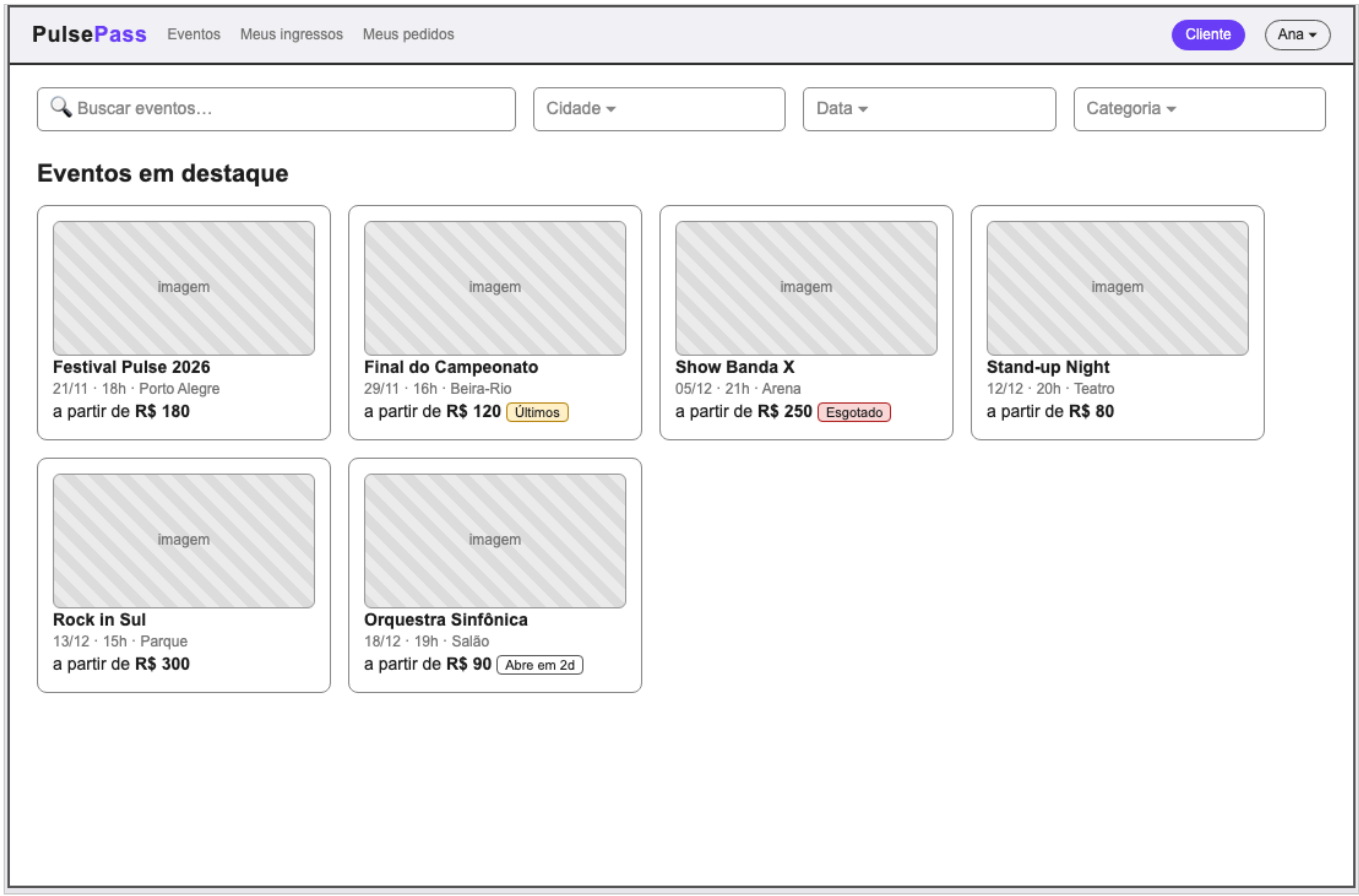
Acesso a rota sem o papel exigido → tela "Acesso negado" (HTTP 403); sem sessão → redireciona para `/entrar` e retorna à rota original após login.

5.3 Telas — Comprador

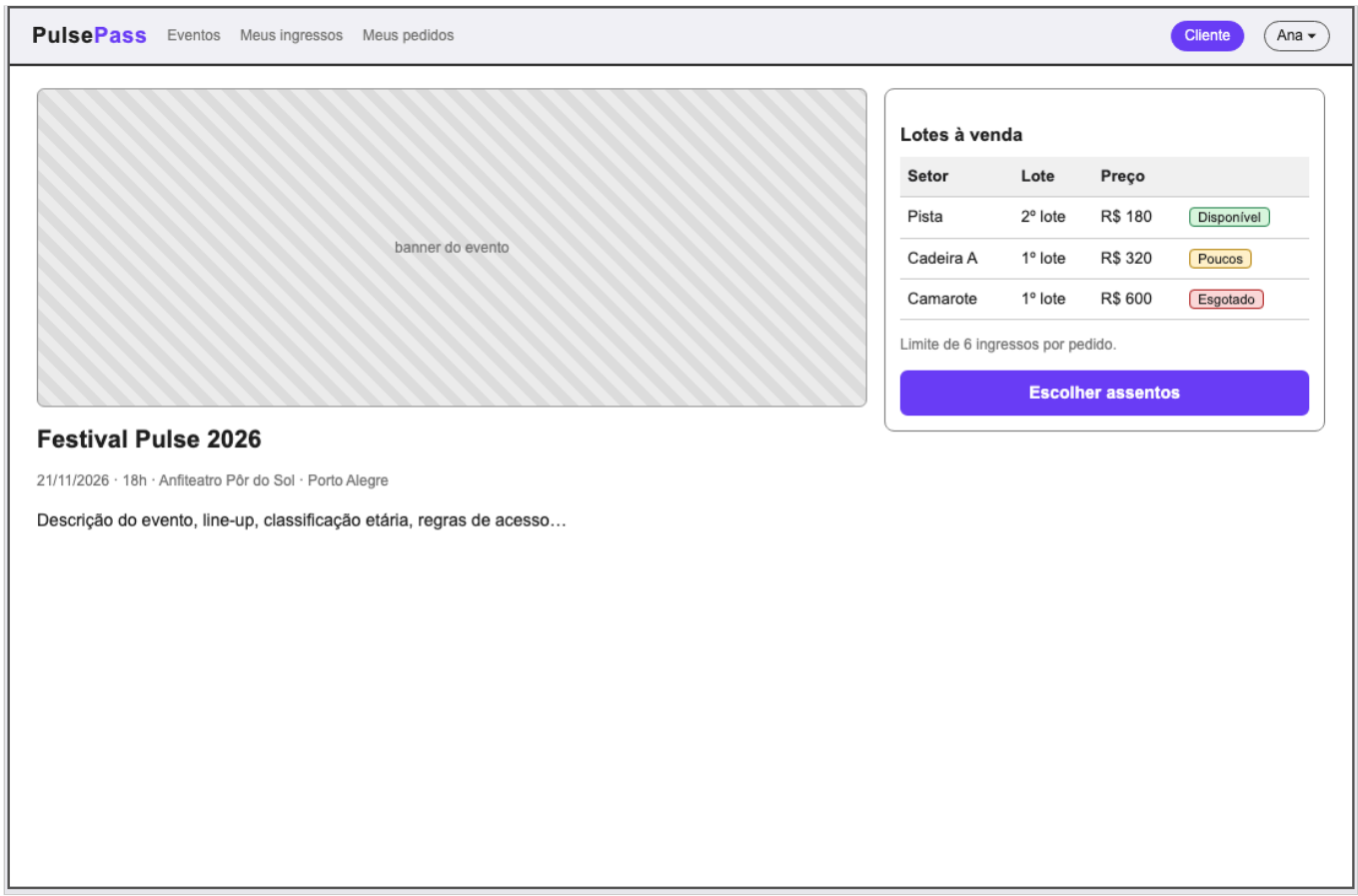
T00 — Login (compartilhada por todos os perfis; autenticação OIDC no Keycloak; estado de erro de credenciais).



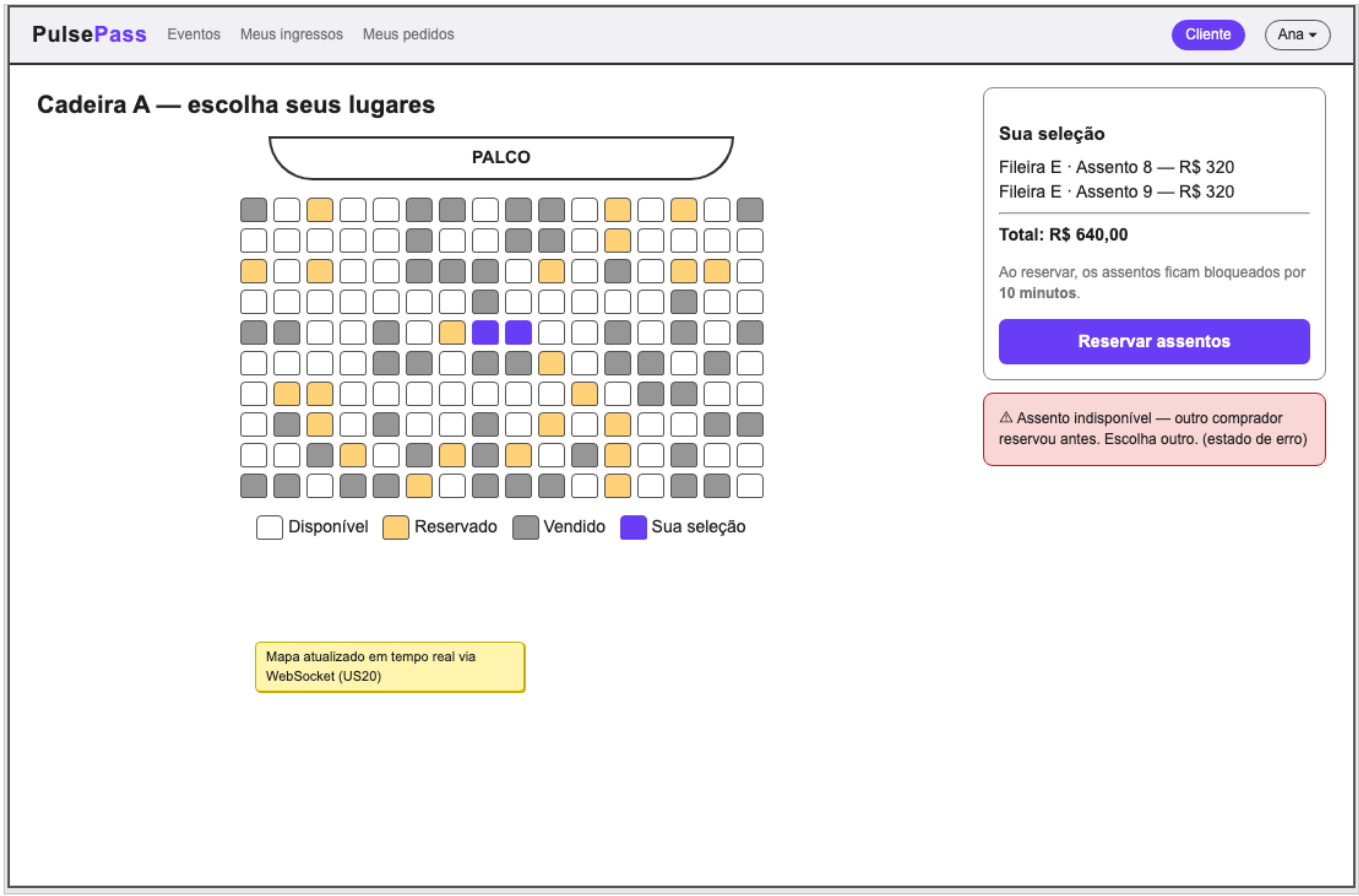
T01 — Catálogo de eventos (busca, filtros, indicadores "Esgotado"/"Últimos"/"Abre em").



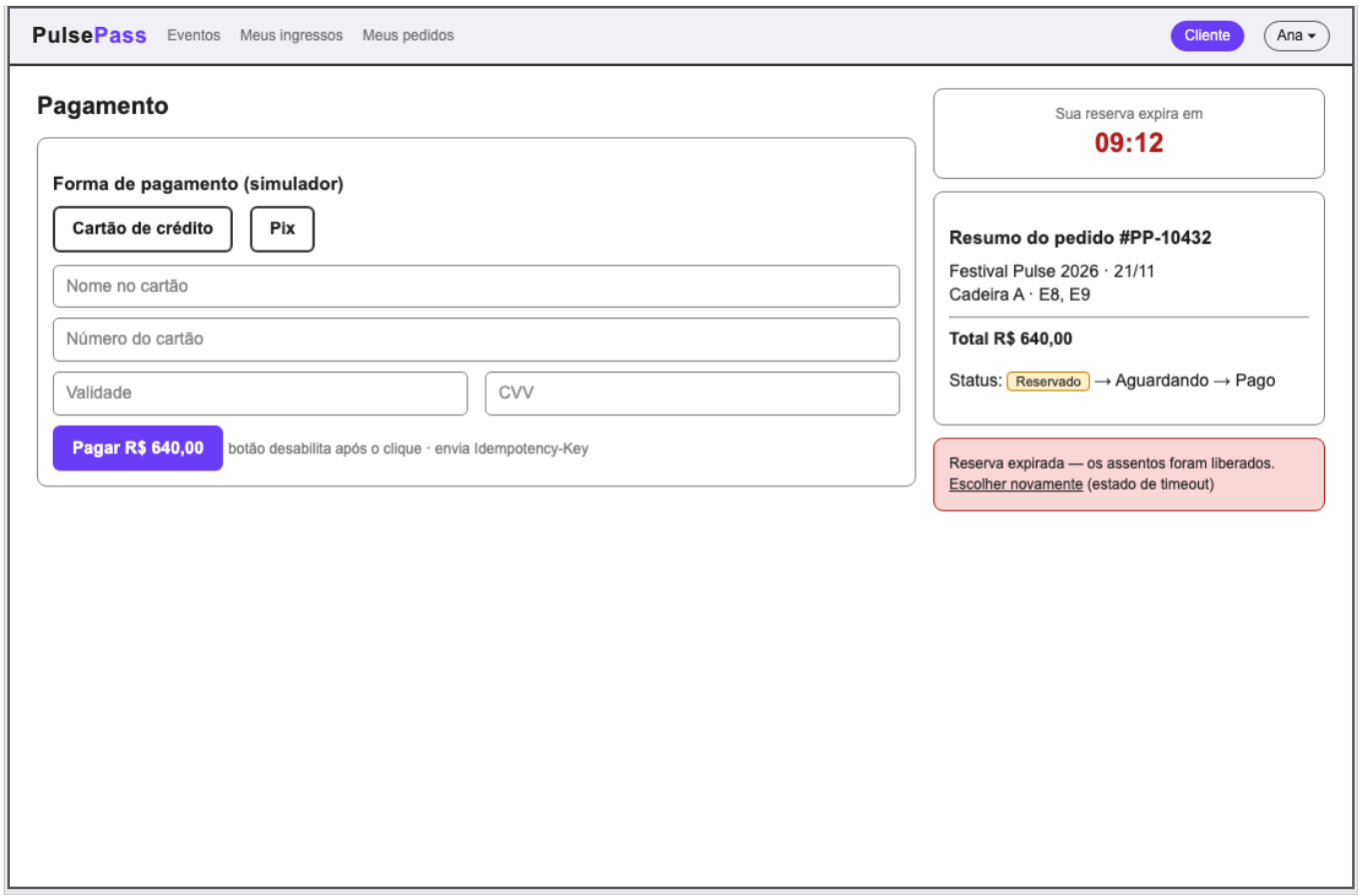
T02 — Detalhe do evento (lotes à venda, limite por pedido).



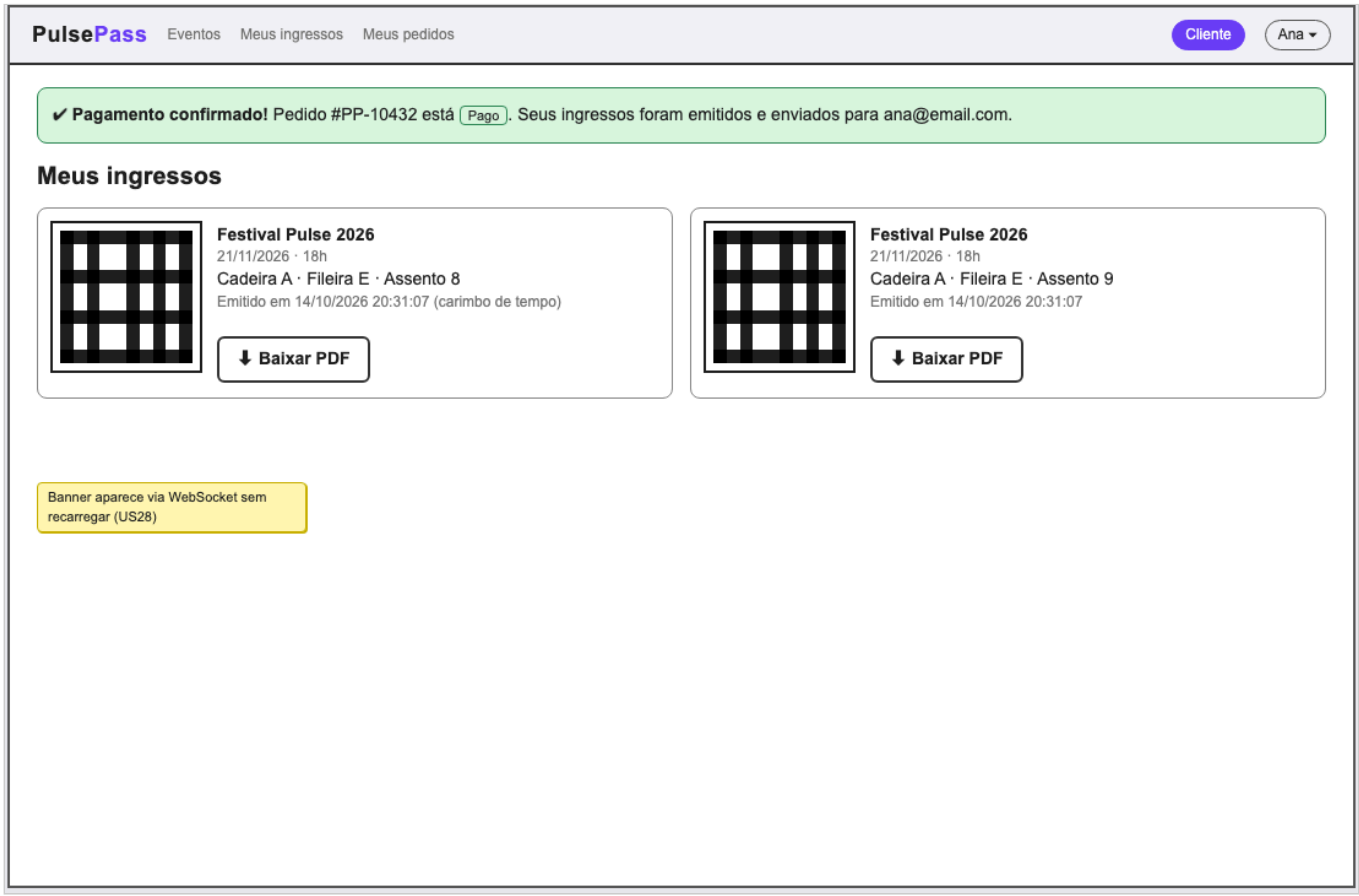
T03 — Mapa de assentos (legenda de status, seleção, aviso de bloqueio de 10 min, erro de assento disputado).



T04 — Checkout (contagem regressiva, status da FSM, botão com Idempotency-Key, estado de reserva expirada).

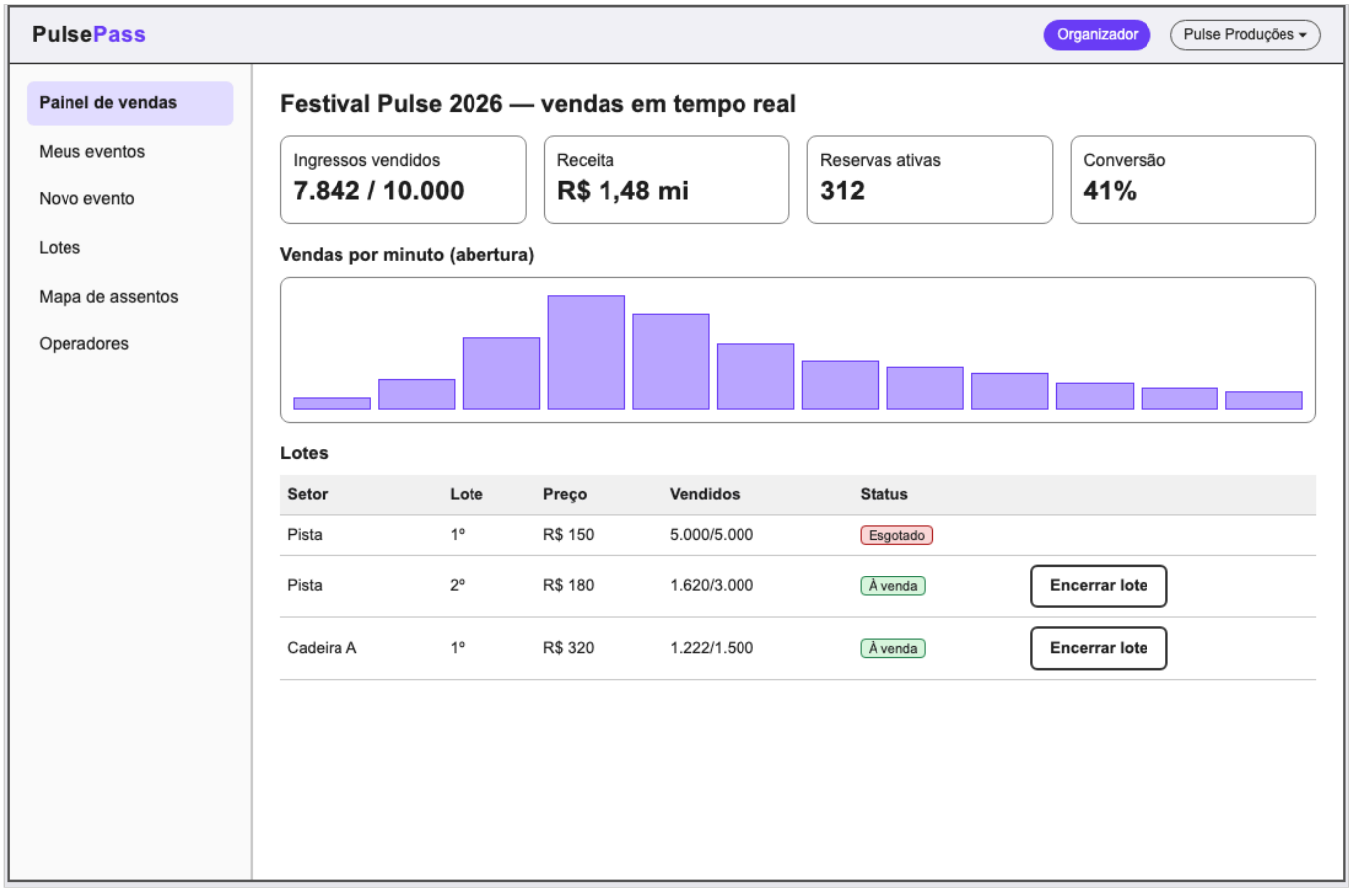


T05 — Confirmação e Meus ingressos (QR Code, carimbo de tempo, download do PDF, aviso em tempo real).

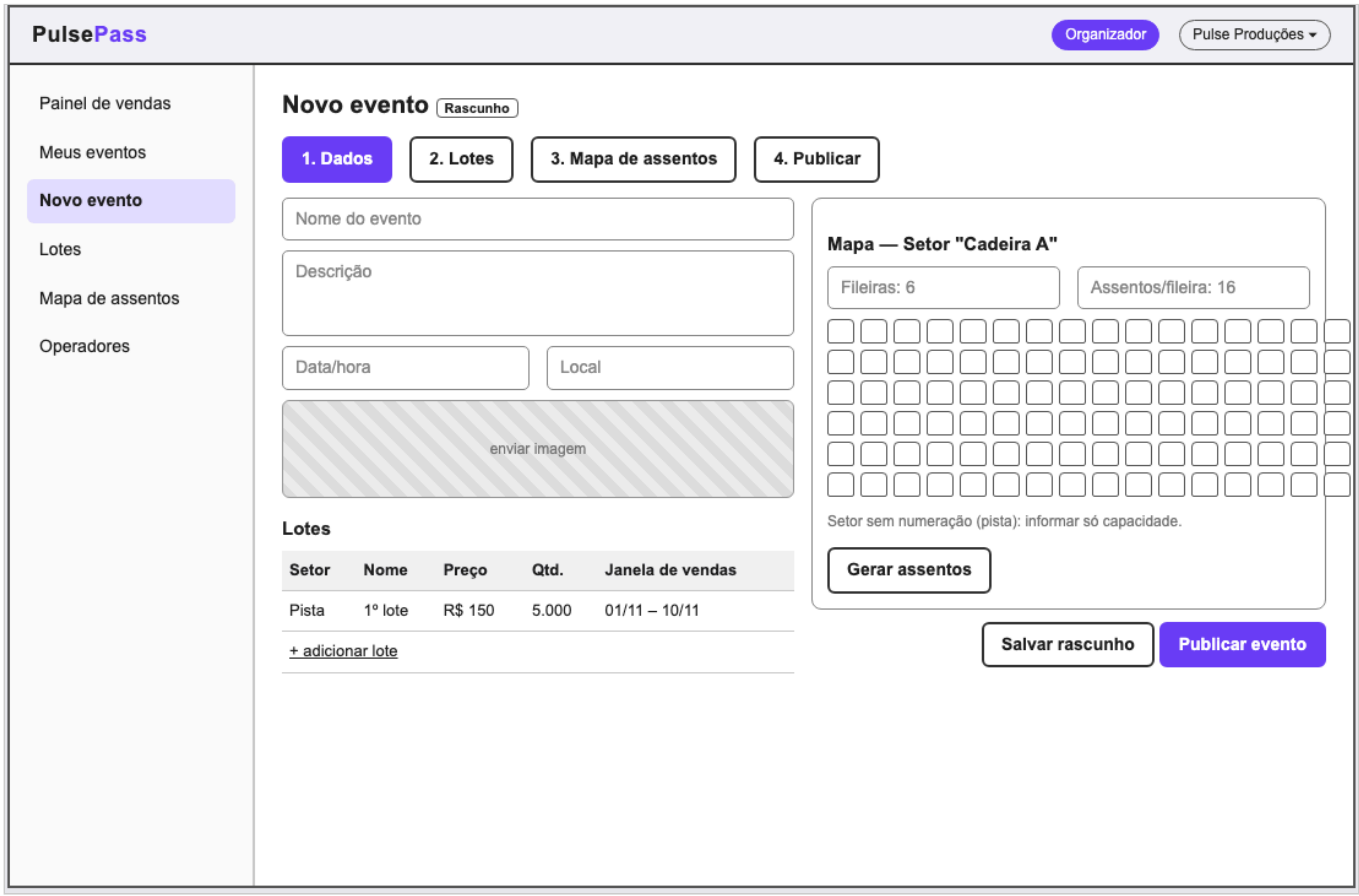


5.4 Telas — Organizador

T06 — Painel de vendas em tempo real (KPIs, vendas por minuto, lotes).

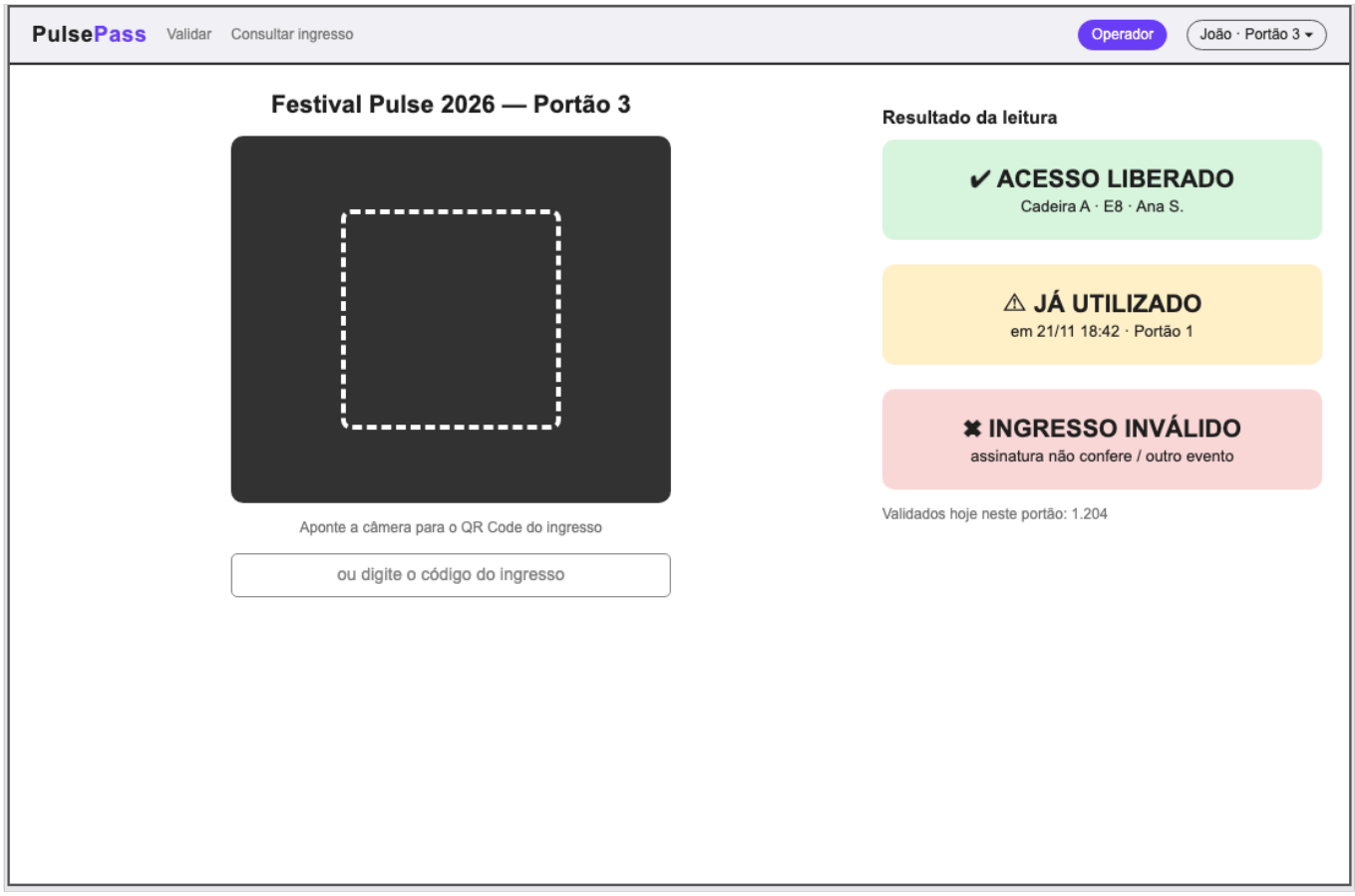


T07 — Criar evento (assistente: dados → lotes → mapa de assentos → publicar).



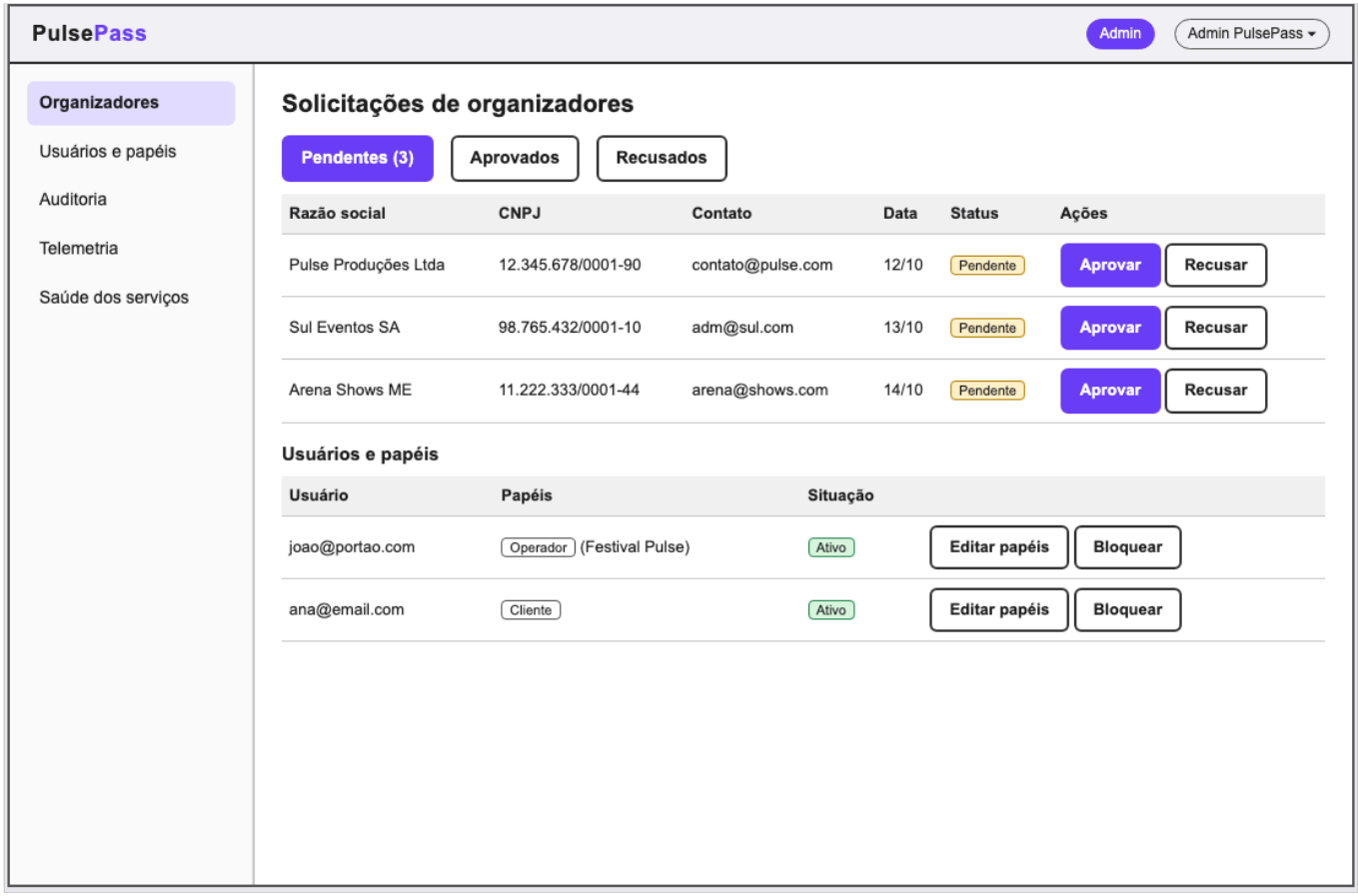
5.5 Telas — Operador

T08 — Validação de acesso por QR Code (câmera do navegador, entrada manual, três resultados possíveis).



5.6 Telas — Admin

T09 — Organizadores e usuários/papéis (aprovação de credenciamento, RBAC, bloqueio).



T10 — Telemetria, saúde dos serviços e auditoria de credenciais.

